CrackVisionX: A Fine-Tuned Framework for Efficient Binary Concrete Crack Detection

Abdulrahman A. ALKannad, Ahmad AL Smadi, Moeen AL-Makhlafi, Shuyuan Yang, *Senior Member, IEEE*, and Zhixi Feng, *Member, IEEE*

***Abstract*— Cracks are critical defects in concrete structures, traditionally identified through human inspection. However, computer vision techniques, especially convolutional neural networks (CNNs), offer promising solutions for automated detection. Driven by this trend, this study proposes *CrackVisionX*, a state-of-the-art deep learning framework for classifying binary concrete cracks. *CrackVisionX* lies in its integration of advanced CNN architectures, ResNet50, MobileNet_v3_large, DenseNet121, and EfficientNetB0, with extensive hyper-parameter tuning. This integration optimizes crack detection accuracy while maintaining low model complexity and reducing bias, making it suitable for real-time applications. Furthermore, the framework introduces a robust data augmentation strategy that effectively addresses dataset imbalances, enhancing model generalization across diverse domains. Additionally, *CrackVisionX* employs comprehensive preprocessing on the METU and SDNET2018 datasets to create six domains: Bridge Deck, Wall, Pavement, SDNET2018, METU, and METU & SDNET2018. The framework's performance is thoroughly evaluated and benchmarked against state-of-the-art methods, utilizing diverse metrics to improve the detection of cracks in concrete structures. EfficientNetB0, a core component of the framework, demonstrated superior performance with exceptional test accuracies of up to 99.71%, 99.78%, 99.55%, 99.89%, 99.98%, and 99.92% for Bridge Deck, Wall, Pavement, SDNET2018, METU, and METU & SDNET2018, respectively. Moreover, we evaluated the robustness of *CrackVisionX* using images contaminated with different types and intensities of noise, demonstrating its reliability and effectiveness. This balance between high accuracy and computational efficiency confirms the framework's potential for practical deployment. The experimental results emphasize the transformative potential of deep learning in construction safety and structural health monitoring.**

***Index Terms*— Concrete crack detection, binary classification, deep learning, CrackVisionX, fine-tuning, structural health monitoring.**

Received 23 July 2024; revised 20 December 2024; accepted 23 February 2025. Date of publication 9 April 2025; date of current version 1 July 2025. This work was supported in part by the National Natural Science Foundation of China under Grant U22B2018 and Grant 62171357. The Associate Editor for this article was M. Guo. *(Corresponding author: Shuyuan Yang.)*

Abdulrahman A. ALKannad, Shuyuan Yang, and Zhixi Feng are with the Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education, School of Artificial Intelligence, Xidian University, Xi'an 710071, China (e-mail: alkannad2021@stu.xidian.edu.cn; syyang@xidian.edu.cn; zxfeng@xidian.edu.cn).

Ahmad AL Smadi is with the Department of Data Science and Artificial Intelligence, Zarqa University, Zarqa 13110, Jordan (e-mail: aalsmadi@zu.edu.jo).

Moeen AL-Makhlafi is with the School of Information Engineering, Wuhan College, Wuhan 430212, China (e-mail: 136342024041@whxy.edu.cn).

Digital Object Identifier 10.1109/TITS.2025.3546770

I. Introduction

CONCRETE crack detection plays a vital role in maintaining the safety and structural integrity of concrete constructions. Concrete, commonly used in urban infrastructure such as bridges, roads, and buildings, is susceptible to cracking due to various internal and environmental factors [1]. Cracks in concrete can lead to structural degradation, including damp damage, holing, and sublayer damage, which compromise the material's durability and pose significant safety risks [2]. Consequently, the precise and prompt identification of concrete cracks is essential for mitigating further deterioration and securing the long-term sustainability of transportation infrastructure [3], [4], [5], [6].

Conventional visual inspection by human experts is commonly employed to assess the condition of concrete civil infrastructure [7]. However, this method presents several limitations, such as being time-intensive, expensive, unsafe, and unreliable in various real-world scenarios [8], [9].To overcome these limitations, researchers have adopted machine learning techniques, such as Random Forest (RF), K-Nearest Neighbor (KNN), Artificial Neural Networks (ANN) [10], and Support Vector Machine (SVM) [11], [12]. Despite advancements in AI-based methods, several challenges remain. First, complex feature extraction processes and the need for large-scale labeled datasets can hinder operational efficiency. Additionally, the fine-tuning process of pre-trained models requires careful consideration to ensure optimal performance. These challenges motivate the development of an advanced AI-based framework that overcomes the limitations of manual and visual inspection methods, providing a more dependable and efficient solution for detecting cracks in concrete structures.

In recent years, deep learning approaches, including the fine-tuning technique, have demonstrated great potential in various domains, including concrete analysis [7] and remote sensing [13], [14]. This technique comprises training a pre-trained model, initially trained on an alternative dataset, by adjusting the weights of some or all layers based on the new dataset. In this paper, we propose an advanced AI-based framework named CrackVisionX for detecting cracks in concrete structures. CrackVisionX framework utilizes convolutional neural networks (CNNs) enhanced with fine-tuning techniques to accurately classify input images as either cracked or non-cracked. By leveraging pre-trained

1558-0016 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

CNN models and training on new concrete crack datasets, CrackVisionX addresses the limitations of manual and visual inspection methods, providing a more robust and effective solution for detecting concrete cracks.

In CrackVisionX framework, the fine-tuning process includes initializing the fine-tuning layers with pre-trained weights obtained from deep learning models trained on the ImageNet dataset [15], which comprises millions of images spanning 1000 different classes.

The primary contributions of this study include:

1) We propose *CrackVisionX*, an advanced AI-based framework that leverages convolutional neural networks (CNNs) for the binary classification of concrete cracks. This framework integrates top-tier models such as ResNet50, MobileNet_v3_large, DenseNet121, and EfficientNetB0, employing hyperparameter optimization to enhance detection accuracy and maintain computational efficiency, all while simultaneously reducing model complexity and bias.
2) We utilized a robust data augmentation strategy to address limitations and imbalances in the datasets, which improved the model's generalization capability. Comprehensive preprocessing techniques were also applied to the METU and SDNET2018 datasets, creating six domains: Bridge Deck, Wall, Pavement, SDNET2018, METU, and combined METU & SDNET2018. To ensure reliable model training and evaluation, each domain was randomly partitioned into 90% training and validation (with 80% for training and 10% for validation) and 10% for testing.
3) We extensively evaluated *CrackVisionX* and compared its performance with state-of-the-art approaches. This evaluation utilized various metrics, assessing its effectiveness in detecting cracks in concrete structures. Performance evaluations affirm the exceptional proficiency of CrackVisionX in detecting concrete cracks. Notably, EfficientNetB0 exhibited superior performance, achieving remarkable test accuracies of up to 99.71%, 99.78%, 99.55%, 99.89%, 99.98%, and 99.92% for the Bridge Deck, Wall, Pavement, SDNET2018, METU, and combined METU & SDNET2018 domains, respectively. Furthermore, we assessed the CrackVisionX robustness by applying images contaminated with different types and intensities of noise, showing its consistency and efficiency.

The paper is organized as follows: Section II reviews the related literature, focusing on advancements in concrete crack detection and deep learning. Section III details the proposed methods, including the crack detection method design, the CNN models with fine-tuning, and the architecture of *CrackVisionX*. Section IV covers the experimental setup and analysis, including the implementation details, data acquisition and distribution, evaluation metrics, and experimental results. Section V presents and discusses the findings, summarizing the study's main results. Finally, Section VI provides the paper's conclusion.

II. Related Work

This section reviews research on concrete crack detection using deep learning, focusing on Convolutional Neural Networks (CNNs). It covers key studies, comparative analyses, and advancements in hybrid optimization and multi-damage recognition.

A. Deep Learning Techniques for Concrete Crack Detection

The integration of deep learning techniques, especially through transfer learning with pre-trained networks, has significantly advanced the domain of concrete crack detection [16], [17]. Sharma et al. [18] utilized well-known architectures such as AlexNet, GoogLeNet, and ResNet18 on the SDNET2018 dataset, comprising over 56,000 images captured under diverse conditions, to enhance the detection of cracks in concrete structures. Their study showcased the superior effectiveness of GoogLeNet and ResNet18 in identifying cracks in bridge decks and walls. Golding et al. [19] also adopted a deep learning-centric approach for crack identification, emphasizing the substantial potential of deep learning in this area. Furthermore, Zhang et al. [20] explored the ConvNeXt method combined with image processing for detecting building cracks, reporting superior performance compared to traditional methodologies.

Tianyong Jiang et al. [21] introduced YOLOMF, a lightweight object detection model designed for multi-damage recognition in concrete bridges. This model integrates MobileNetV3 with YOLOv4, achieving high detection speed (85 frames per second) while maintaining good performance under motion blur, fog, and changing lighting conditions. However, the model's reliance on high-quality data and computational resources limits its scalability in resource-constrained environments. This highlights the need to balance detection accuracy and computational efficiency in real-world applications, especially large-scale monitoring tasks.

Liu et al. [22] proposed a lightweight network bridge multi-damage recognition. The model demonstrated strong capabilities in detecting cracks and other defects in noisy, challenging conditions, achieving an 85.06% increase in detection speed and reducing inference time. This makes it suitable for real-time monitoring on mobile or edge devices. This approach offers an important trade-off between model complexity and computational efficiency, especially for deployment on resource-constrained platforms.

Hou et al. [3] leveraged MobileNet and MobileNet-SSD for distress detection in asphalt and concrete infrastructures, achieving 97.8% accuracy. Despite their efficiency, challenges like environmental variations and dataset diversity persist, requiring further improvements.

Yu et al. [23] optimized a deep CNN for crack detection in concrete structures by incorporating the Enhanced Chicken Swarm Algorithm (ECSA). This hybrid approach improved both the accuracy and computational efficiency compared to traditional CNNs. However, the model's dependence on high-quality images and its potential for overfitting may limit its generalizability in diverse environments. Wan et al. [24] introduced a refined Particle Swarm Optimization (PSO)

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

algorithm to improve crack detection in building walls, particularly for irregular crack shapes. This approach showed superior performance but lacked scalability for real-time applications, which is crucial for large-scale monitoring systems. These advancements illustrate promising solutions for improving detection accuracy and efficiency, though challenges related to model generalization and real-time scalability remain.

Amirkhani et al. [25] systematically reviewed deep learning applications for visual concrete bridge defect classification and detection. They utilized CNNs such as ResNet, DenseNet, InceptionV3, and EfficientNetV2 to identify defects like cracks, spalling, and corrosion stains. Fine-tuned models achieved high accuracy, precision, and recall, outperforming traditional methods. Additionally, object detection algorithms like YOLO and Mask R-CNN enabled real-time defect localization, supporting automated inspections. However, limitations remain, including challenges with low-contrast defect detection, sensitivity to environmental conditions, and reliance on extensively annotated datasets.

B. Comparative Analysis and Fine-Tuning of Deep Learning Models

To assess the latest deep learning approaches for crack detection, it is important to conduct a comparative analysis of different models, considering architecture, dataset size, transfer learning, and deployment challenges. This analysis evaluates performance and identifies strengths and limitations in real-world applications.

Fine-tuning pre-trained models have gained traction due to their ability to accelerate training and improve performance with smaller datasets. ermet and Paal [26] introduced a hybrid model combining Vision Transformers (ViTs) and CNNs, achieving accuracies of 96.41%, 92.76%, and 92.81% for road, wall, and bridge cracks, respectively. While ViTs capture global contextual features, they require large datasets and substantial computational resources, limiting their real-time application. This trade-off between accuracy and computational cost is a key challenge for models relying on ViTs.

Similarly, Bukhsh et al. [27] investigated the use of cross-domain transfer learning with pre-trained models, such as ResNet50, for crack detection. While transfer learning facilitated faster convergence, the approach encountered significant challenges when the source and target datasets were not sufficiently aligned.

MobileNet-based fine-tuning has emerged as a promising approach for real-time crack detection due to its relatively low computational requirements. Dais et al. [28] highlighted MobileNet's effectiveness in detecting cracks in masonry walls, noting its ability to balance accuracy and computational efficiency. However, MobileNet's performance tends to be inferior to that of deeper models like ResNet, particularly when fine details of cracks are involved, such as in complex scenarios. In contrast, Yongqing Jiang et al. [29] improved the MobileNet-SSD model, demonstrating that smaller models can deliver satisfactory results with minimal computational overhead, making them particularly well-suited for resource-constrained settings. This emphasizes the trade-off between deployment efficiency and detection performance. At the same time, lighter models like MobileNet offer efficiency but may not capture the fine details as effectively as more complex models.

Zarski et al. [30] applied pruning to convolutional neural networks combined with fine-tuning to optimize models for real-time edge-device deployment. Pruning reduced model size and computational complexity but at the cost of decreased accuracy, particularly for detecting fine cracks.

Dorafshan et al. [31] used AlexNet for crack detection with the SDNET2018 dataset. They found that fine-tuning AlexNet worked well for early-stage crack detection but was less effective for more complex or irregular crack patterns. In contrast, deeper models like ResNet and DenseNet showed better performance in such cases, but they require more computational power, limiting their use in real-time applications.

Gopalakrishnan et al. [32] were among the first to apply VGG-16 for crack detection on concrete and asphalt surfaces, showing that it can achieve high accuracy even with smaller datasets. However, VGG-16's performance drops when dealing with more complex or large-scale datasets. This suggests that while simpler models can work effectively for smaller datasets, they may struggle to scale for larger, real-world datasets, where deeper models like ResNet are more suitable. Similarly, Zheng et al. [6] employed pre-trained models such as YOLOv3 and RetinaNet for crack detection on railway tracks. While YOLOv3 excels in real-time detection, it often falls short in detecting finer cracks compared to more complex models like ResNet.

Philip et al. [33] compared several CNN architectures, including AlexNet, VGG, and ResNet, for crack detection. Deeper architectures like ResNet50 and DenseNet121 excelled in detecting fine cracks but come with higher computational costs, which limits their deployment in resource-constrained environments. This again illustrates the need for careful selection based on the available computational resources and the complexity of the cracks being detected.

Fine-tuning pre-trained models for crack detection comes with several challenges, the most notable being the quality and diversity of training datasets. Many models are trained on small, controlled datasets that may not fully represent the complexity of real-world scenarios. Expanding these datasets and employing data augmentation techniques can help improve the models' generalization ability to diverse conditions. Another challenge is the computational cost of training larger models like ResNet and DenseNet. To address this, researchers are exploring lightweight models that balance performance and efficiency. Finally, robustness to environmental conditions remains a significant concern. Models trained in controlled environments often struggle with real-world challenges, such as varying lighting conditions or weather changes.

In contrast to previous studies that relied on smaller datasets, our research leverages two large-scale datasets, METU and SDNET2018, containing over 96,000 images captured in various environmental conditions. This diversity allows the model to better generalize across different cracks and structural scenarios, making it more effective for real-world monitoring applications.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

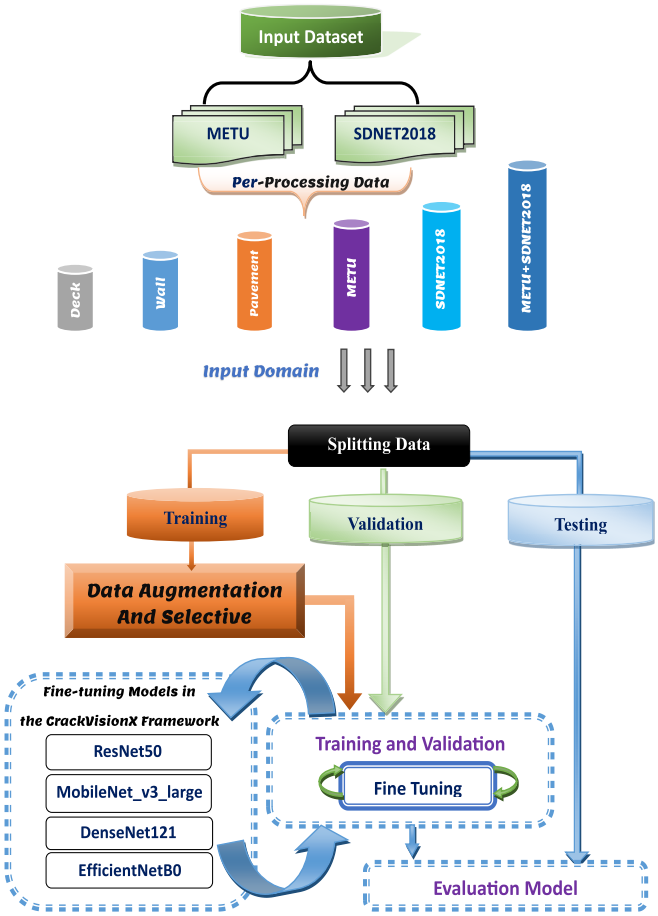


Fig. 1. Crack detection method design.

III. Proposed Method

This section outlines our proposed methodology for crack detection. We elaborate on the crack detection method design and then detail CNN models with fine-tuning. Finally, we present our newly developed model.

A. Crack Detection Method Design

In designing our crack detection method, we pinpointed four key areas for improvement: input image processing, dataset diversity, pre-processing, and Fine-Tuning model selection, as shown in Fig. 1. The following steps highlight the essential components of our methodology:

- We utilized a binary classification methodology to differentiate images into cracked or uncracked categories. This simplification helps the model effectively learn the distinctive features of images to classify them.
- To enhance dataset diversity, we gathered samples from multiple commonly used databases. This strategy ensured that our dataset included various concrete structures, crack patterns, and environmental conditions, allowing the model to generalize well. Moreover, we divided the dataset into training, validation, and test sets, stratified by each class, i.e., domain, to accurately evaluate model performance.
- We placed particular emphasis on pre-processing, which is often overlooked in previous studies. We carefully pre-processed the dataset to minimize background noise and applied realistic data augmentation techniques. Additionally, we increased the diversity and size of the training set, enhancing the model's robustness and generalization capability.
- We employed a fine-tuning approach to select the most suitable model architecture, using cutting-edge models like ResNet50, MobileNet_v3_large, DenseNet121, and EfficientNetB0. Fine-tuning these models with our pre-processed dataset allowed us to determine the model that achieved the highest performance and accuracy in detecting concrete cracks.

B. CNN Models With Fine-Tuning

This section explores fine-tuning pre-trained Convolutional Neural Network (CNN) models for crack detection. Fine-tuning comprises adjusting the parameters of models like ResNet50, MobileNetV3-Large, DenseNet121, and EfficientNetB0 to optimize their performance for this specific task.

1) ResNet50: ResNet50 is a deep CNN architecture comprising 50 layers and incorporating residual connections to alleviate the vanishing gradient problem. Fine-tuning involves replacing the final fully connected layer to align with the target classes, unfreezing all layers, and training on the fine-tuning dataset. Based on validation performance, the best model is saved.

2) MobileNetV3-Large: MobileNetV3-Large is designed for efficient image classification on resource-constrained devices, using techniques like depthwise separable convolutions and squeeze-and-excitation blocks. It undergoes fine-tuning by updating the final fully connected layer to match the target classes. All layers are then unfrozen and trained on the fine-tuning dataset. The model demonstrating the highest validation performance is saved.

3) DenseNet121: DenseNet121 is a deep CNN architecture featuring 121 layers and dense connections. The fine-tuning process involves substituting the final fully connected layer to align with the target classes, unfreezing all layers, and training on the fine-tuning dataset. The best-performing model is saved based on validation performance.

4) EfficientNetB0:: EfficientNetB0 is the smallest variant in the EfficientNet series, designed to balance network depth, width, and resolution. For fine-tuning, the final fully connected layer is replaced to suit the target classes, all layers are unfrozen, and training is conducted on the fine-tuning dataset. The model with the highest validation performance is then saved.

Table I summarizes these four models by highlighting their key differences, strengths, and applications in crack detection. Additionally, it details their distinct architectural features and suitability for this task.

In our experiments, we initially explored partial layer unfreezing, where only selected layers were trained while others remained fixed. However, this approach limited the model's ability to learn crack details, resulting in lower accuracy, especially under varying lighting and texture conditions. On the other hand, we found that full unfreezing consistently led to superior performance in crack detection, a task sensitive to the

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

TABLE I
CORE DIFFERENCES BETWEEN THE FOUR MODELS FOR CRACK DETECTION

Model	Architectural Features	Strengths and Applications
ResNet50 [34]	Residual connections (skip connections)	Robust in deep networks and effective in mitigating the vanishing gradient problem. Ideal for extracting complex features in fine-grained tasks, such as crack detection in detailed images.
MobileNetV3-Large [35]	Depthwise separable convolutions, lightweight design	Optimized for mobile devices and real-time applications, offering low latency and high efficiency. Particularly well-suited for crack detection in embedded systems or resource-limited environments, providing fast and efficient performance.
DenseNet121 [36]	Dense connectivity (each layer connected to all previous layers)	Efficient feature reuse, fewer parameters, and strong performance in tasks requiring detailed feature extraction. Particularly useful for detecting fine cracks where high-resolution feature maps are critical.
EfficientNetB0 [37]	Compound scaling (depth, width, resolution)	Efficient in terms of parameters with high scalability. Well-suited for deployment in resource-constrained environments, such as mobile devices, enabling efficient real-time crack detection.

dataset's nuanced texture, lighting, and structural variations. This approach also allowed each model to fully adjust its feature extraction, thereby adapting to the diverse patterns and environmental conditions in our crack detection images. Thus, unfreezing all layers was essential to achieve the highest accuracy and robustness across our dataset.

Although unfreezing all layers requires more computational cost and increases training time and the risk of overfitting, It allows models to fully adapt to the dataset's unique crack patterns and environmental conditions. To mitigate longer training times, we used pre-trained models (e.g., ResNet50, EfficientNetB0) and implemented early stopping and learning rate scheduling to optimize training efficiency. Furthermore, we used dropout layers in the custom head architecture, introducing randomness by deactivating neurons during training. This further reduced overfitting and enhanced the model's ability to capture subtle features within cracks. In addition, by carefully managing the complexity of these custom head designs, we balanced model performance and computational efficiency, ensuring that the sophisticated architecture provides significant advantages without incurring prohibitive costs. These combined strategies enhance CrackVisionX's accuracy and robustness while maintaining manageable training times and computational requirements.

C. Architecture of CrackVisionX

We propose CrackVisionX, a crack detection model that employs a customized Convolutional Neural Network (CNN) architecture. This architecture is specifically designed to fulfill the requirements of real-world crack detection applications, making it highly suitable for practical use. CrackVisionX architecture is based on fine-tuning models in which a diverse set of data augmentation techniques is employed. Then, all layers are fine-tuned to select the most suitable model. Finally, the data is processed using a customized head architecture consisting of four distinct layers to replace the original classification layer, as demonstrated in Fig. 2.

Based on extensive image processing and data augmentation steps, the CrackVisionX model employs a customized Convolutional Neural Network (CNN) architecture to detect and classify cracks. The ReLU function, a type of rectified linear unit [38], is employed as the activation function in the CNN layers, as illustrated in Equation 1:

$$f(x) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{if } x \geq 0 \end{cases} \tag{1}$$

A batch normalization (BN) layer is added between each convolution layer and activation function layer (ReLU) to ensure data integrity and accelerate training efficiency.

1) Customized Head Architecture: To enhance the accuracy of the CrackVisionX model, a customized head was proposed, consisting of the following layers:

1) Linear Layer

$$y = W_1 \cdot x + b_1 \tag{2}$$

2) Dense Layer with ReLU Activation

$$h = \text{ReLU}(W_2 \cdot y + b_2) \tag{3}$$

where $\text{ReLU}(z) = \max(0, z)$.

3) Dropout Layer

$$T_{\text{dropout}}(h) = \begin{cases} 0 & \text{with probability } p \\ \dfrac{h}{1-p} & \text{with probability } 1-p \end{cases} \tag{4}$$

4) Sigmoid Layer

$$\sigma(z) = \frac{1}{1+e^{-z}} \tag{5}$$

where z represents the input value.

These layers, represented through their equations, provide the core transformations and operations used in the CrackVisionX model to detect and classify cracks with high accuracy and robustness.

In a word, CrackVisionX is a fine-tuned CNN model optimized for crack detection by updating the final fully connected layer to align with the target classes, unfreezing all layers, and training on the fine-tuning dataset. The model with the highest validation performance is saved. To improve accuracy, we replaced the original classification layer with a customized head consisting of a linear layer, dense layer, dropout layer, and sigmoid layer, as shown in Fig 2. These layers are configured according to the features detailed in Table II.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

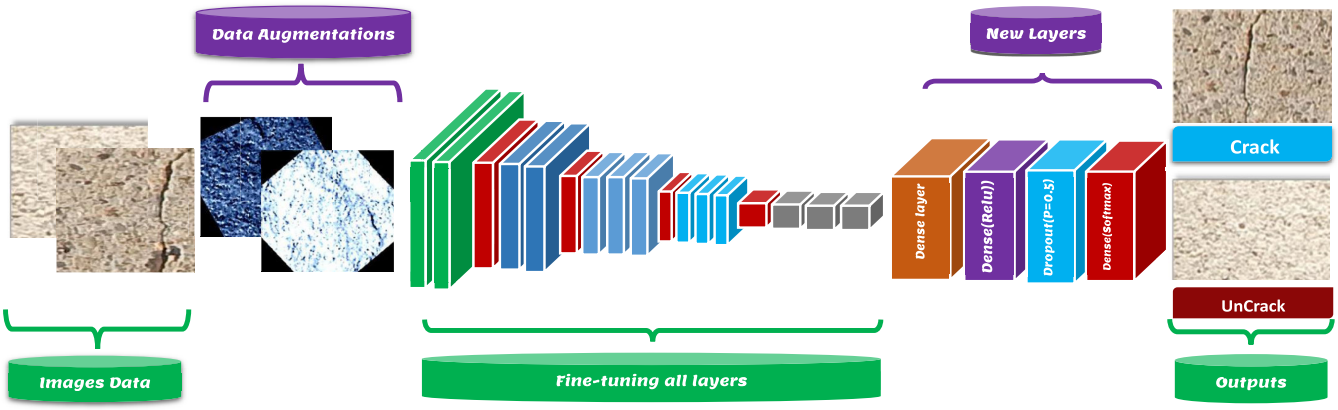


Fig. 2. CrackVisionX architecture based on fine-tuning models.

TABLE II
MODEL ARCHITECTURE AND PARAMETERS

Model	Layer	Type	Features	Linear	Parameters
ResNet50	1	Linear	2048	2048	**27,708,482**
	2	ReLU	2048	-	
	3	Dropout (p=0.5)	-	-	
	4	Linear	2	2	
MobileNetV3-Large	1	Linear	1280	1280	**5,844,274**
	2	ReLU	1280	-	
	3	Dropout (p=0.5)	-	-	
	4	Linear	2	2	
DenseNet121	1	Linear	1024	1024	**8,005,506**
	2	ReLU	1024	-	
	3	Dropout (p=0.5)	-	-	
	4	Linear	2	2	
EfficientNet-B0	1	Linear	1280	1280	**5,649,790**
	2	ReLU	1280	-	
	3	Dropout (p=0.5)	-	-	
	4	Linear	2	2	

TABLE III
DATA AUGMENTATION TECHNIQUES APPLIED DURING TRAINING AND VALIDATION PHASES

Augmentation Technique	Training	Selective for Cracked Images
Resizing	224 x 224	224 x 224
Random Horizontal flip	p=0.5	p=0.5
Random Vertical flip	p=0.5	p=0.5
Random Sharpness adjustment	factor=2, p=0.5	factor=2, p=0.5
Random Autocontrast	p=0.5	p=0.5
Random Grayscale	p=0.5	p=0.5
Random Rotation	45 degrees	45 degrees
Color jittering	N/A	bright=0.2, contrast=0.2, hue=0.1,saturation=0.2
Tensor Transformation	Applied	Applied
Normalization	Applied	Applied
Validation Augmentation		
Resizing		Applied
Normalization		Applied

These architectural modifications address the challenges of overfitting, generalization, and performance encountered in crack detection research.

D. Data Preprocessing and Augmentation

1) Data Preprocessing: To enhance model performance and facilitate effective learning, we applied a series of preprocessing pipelines in the CrackVisionX framework, carefully designed to ensure consistent input quality, as reported in Table III. Firstly, each input image is resized to a standardized resolution of 224×224 pixels. This resizing ensures compatibility with convolutional neural network (CNN) architectures and reduces computational complexity, facilitating efficient training and inference processes. Following resizing, images are converted into PyTorch tensors with pixel intensity values scaled to the range [0,1]. This normalization step is crucial for maintaining uniformity across datasets and stabilizing the training process.

Furthermore, pixel intensity values are normalized using standard ImageNet statistics. This normalization mitigates biases caused by varying lighting conditions. It ensures that the input data conforms to the distribution expected by pre-trained models, thereby improving the model's ability to generalize across different datasets. Additional preprocessing steps include noise reduction and contrast enhancement. Noise reduction filters out irrelevant variations in the images, enhancing the clarity and visibility of crack features. Contrast enhancement techniques improve the distinction between cracks, particularly beneficial in challenging environmental conditions where lighting may vary significantly.

For the validation dataset, a simplified preprocessing pipeline is employed to preserve the original structure of the images while maintaining compatibility with the trained model. This approach ensures that the model can effectively generalize to real-world data without introducing additional distortions during validation.

2) Data Augmentation: A comprehensive data augmentation strategy has been employed during the training phase to improve the generalization capability of the CrackVisionX framework and simulate real-world variability. The key augmentation techniques include random rotations within a range of $\pm 45°$, addressing diverse orientations of cracks typically encountered in real-world scenarios, as reported in Table III. Horizontal and vertical flips are applied with a probability of

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

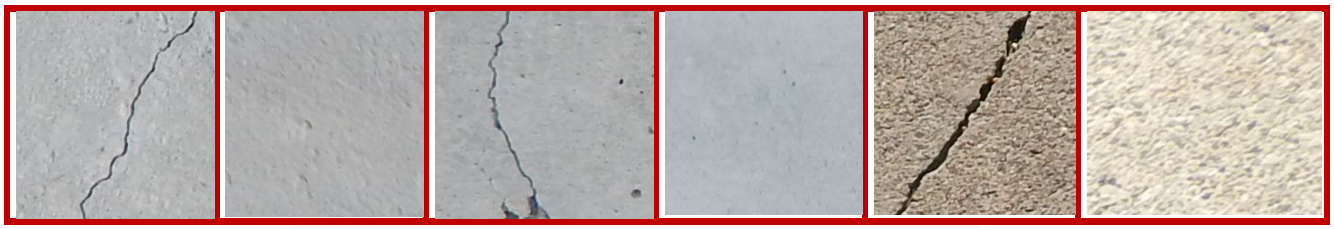


Fig. 3. Original data set: Cracked and uncracked samples.

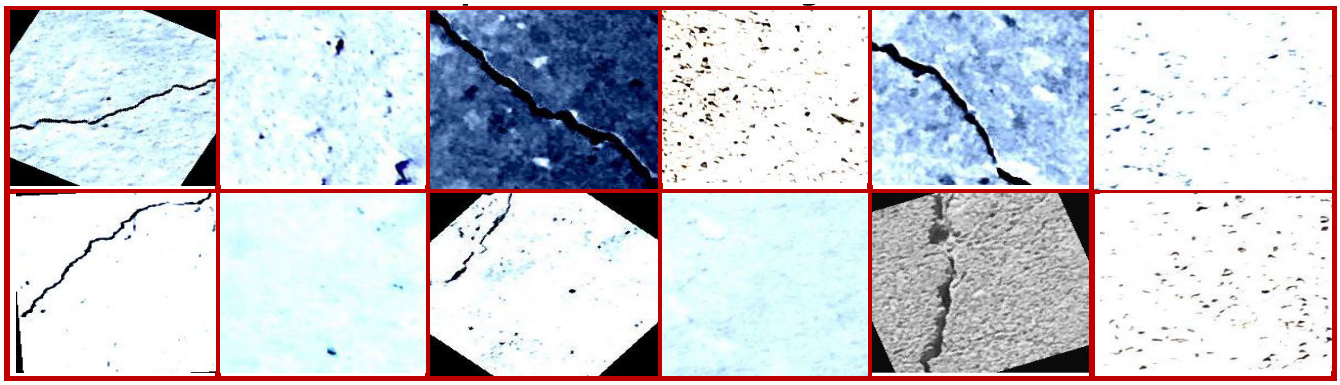


Fig. 4. Data augmentation techniques: random flips, sharpening, autocontrast, grayscale, rotation, and color jitter.

50%, introducing structural variability and simulating diverse perspectives of surfaces. Sharpness adjustments, applied with a probability of 50%, replicate imaging inconsistencies such as blurred or highly detailed regions. Auto-contrast adjustments dynamically enhance crack visibility under varying lighting conditions, improving the robustness of the model to illumination variations.

Additionally, color jittering has been applied to modify brightness, contrast, hue, and saturation, effectively simulating environmental variations such as shadows or lighting shifts. Random grayscale transformations are applied with a probability of 50%, emphasizing structural features while reducing dependence on color information. Controlled noise has been introduced to replicate real-world imaging artifacts, such as graininess and sensor interference, further enhancing the dataset's robustness. For selective classes (e.g., crack images), targeted augmentations such as enhanced color jittering have been applied to increase dataset diversity and enable the model to capture fine-grained distinctions effectively.

To address the class imbalance and mitigate potential overfitting, we implemented selective augmentation techniques specifically for cracked images in our dataset, where uncracked cases vastly outnumber cracked cases. Transformations such as rotation, flipping, scaling, and adding random noise were applied more extensively to cracked samples. This approach enriched the representation of cracked samples within the training set, enhancing the model's ability to detect cracks accurately despite the dataset imbalance. To further mitigate the challenges of class imbalance, we employed targeted augmentations exclusively for cracked images. These augmentations increased the diversity and frequency of cracked samples, reducing the model's bias toward the majority (uncracked) class. Additionally, we utilized a weighted cross-entropy loss function to prioritize the minority class, as defined by Equation 6. This method assigned higher penalties for misclassifications of cracked images compared to uncracked ones, encouraging the model to focus more on identifying cracked instances.

$$\mathcal{L}_{\text{weighted}} = -\frac{1}{N}\sum_{i=1}^{N}\Big[w_1 \cdot y_i \log(p_i) + w_0 \cdot (1-y_i)\log(1-p_i)\Big] \tag{6}$$

Here, N is the total number of samples, y_i the ground truth label (1 for cracked, 0 for uncracked), and p_i the predicted probability for the cracked class. Weights w_1 and w_0, computed as the inverse class frequencies, assign higher penalties to the minority class, balancing training and improving crack detection.

Figs. 3 and 4 present a visual comparison between the original dataset images and their augmented counterparts, illustrating the variety and depth of transformations applied to enrich the dataset.

IV. Experimental Setup

In this section, we detail the implementation steps for the proposed model, describe the datasets used for evaluation, and outline the metrics used for evaluation.

A. Implementation Steps

To leverage the flexibility and dynamic computational graph capabilities, we utilized the PyTorch deep learning environment. The key components of the implementation steps include:

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
SENSITIVITY ANALYSIS OF LEARNING RATE AND BATCH SIZE ON MODEL PERFORMANCE (METU + SDNET2018)

Learning Rate (lr)	Batch Size	Validation Accuracy (%)	AUC
2×10^{-4}	**32**	**99.92**	**0.999**
1×10^{-3}	32	99.25	0.987
2×10^{-5}	32	98.87	0.981
2×10^{-4}	64	99.61	0.990
2×10^{-4}	128	99.22	0.985

1) Normalization and Activation Functions: We implemented data augmentation and fine-tuning methodologies, incorporating batch normalization with batch sizes of 32, 64, and 128 following each convolutional layer to enhance convergence speed during the training process. ReLU activation functions are applied after batch normalization to introduce non-linear relationships between the network layers, effectively mitigating overfitting and gradient issues of vanishing.

2) Optimization Strategy: We utilized the Adam optimizer for our models. The learning rate was initially set to 2×10^{-4}, with a 10-fold reduction every 50 epochs and training durations of 30 and 50 epochs for each batch size. The model was checkpointed at intervals of every 30 and 50 epochs for each batch size, ensuring robust training and avoiding overfitting.

3) Hyperparameter Optimization: The optimization of the hyperparameters was carried out using a grid search procedure over the following ranges:

- Learning Rate: $[2 \times 10^{-5}, 2 \times 10^{-4}, 1 \times 10^{-3}]$
- Batch Size: [32, 64, 128]
- Epochs: [30, 50]

To provide experimental evidence supporting the choice of hyperparameters, we performed the tuning on the METU + SDNET2018 domain (one of the six domains used in our experiments). The grid search results are summarized in Table IV. The best-performing configuration was found with a learning rate of 2×10^{-4}, a batch size of 32, and a training duration of 50 epochs. This configuration yielded a validation accuracy of 99.92%. This setup balanced both training efficiency and accuracy, as evidenced by the results in the METU + SDNET2018 domain.

4) Model Configuration: The model parameters are configured as follows: an input image size of 224×224, a momentum of 0.9, a weight decay of 2×10^{-4}, and a loss weight of 1.0 for every side-output layer as well as the final layer. The model is saved every 50 epochs.

5) Computational Resources: Training and experiments were conducted on the AutoDL platform with an Ubuntu 20.04 OS, using a 10-core Intel Xeon Gold 6148 CPU and various NVIDIA GPUs (RTX A5000, GeForce RTX 3080, TITAN Xp) to ensure robust computational capacity.

6) Sensitivity Analysis: To further evaluate the robustness of the chosen hyperparameters, we conducted a sensitivity analysis by varying the learning rate and batch size. Table IV shows the impact of these variations on model accuracy and AUC. The results demonstrate that slight changes in these hyperparameters result in only minor fluctuations in model performance, confirming the robustness and stability of the selected settings.

B. Acquisition of Data and Distribution

To provide a more comprehensive representation of these factors, we used three datasets in our study. Two of these datasets, SDNET2018 and METU, come from different sources, and we created a third dataset by merging the first two. Afterward, we conducted extensive processing to categorize these datasets into six distinct domains: Bridge Deck, Wall, SDNET2018, METU, and METU + SDNET2018. The two main datasets, SDNET2018 and METU, were selected for their complementary characteristics, which together offer a balanced representation of diverse concrete structures, crack patterns, and environmental conditions:

1) SDNET2018 Dataset: The SDNET2018 dataset, created by Utah State University, features structural cracks ranging from 0.06 mm to 25 mm [31]. This dataset includes 56,092 real-world images of various structural elements such as bridge decks, walls, and pavements, all subject to different environmental exposures. It covers four domains (Deck, Wall, Pavement, and SDNET2018 domain), capturing a wide range of environmental conditions, including variations in lighting, weather, and surface textures. The SDNET2018 dataset is especially valuable for modeling the diversity of crack patterns that arise from natural structural stresses and environmental influences. The dataset was redistributed into four domains:

a) Bridge deck: Features structural elements of Bridge Decks, comprising 13,620 color images. These images capture cracks caused by environmental exposure and structural stresses, providing varied patterns in a real-world setting.

b) Wall: Features structural elements of walls, comprising 18,138 color images. This domain includes cracks formed on vertical surfaces under different conditions such as weathering, erosion, and load-bearing stress, ensuring coverage of various real-world cracking patterns.

c) Pavement: Features structural elements of pavements, comprising 24,334 color images. Pavement cracks, often more complex due to environmental wear, traffic, and surface conditions, are captured under different lighting conditions and environmental factors.

d) SDNET2018: Encompasses structural elements from all the above three domains, comprising 56,092 color images. This combined domain integrates diverse environmental conditions (e.g., different times of day and weather conditions) and various crack patterns from multiple structural components, enhancing the dataset's coverage of real-world scenarios.

Based on the diverse datasets representing these scenarios, our crack detection approach was designed to address these challenges. This allows the model to adapt and perform well across the different application domains.

2) METU Dataset: This dataset represents a domain consisting of 40,000 RGB images derived from 458 high-resolution photographs taken at the Middle East Technical University (METU) in Ankara, Turkey [39], [40]. Unlike SDNET2018, the METU dataset focuses on controlled laboratory conditions, providing clean, high-resolution images of both cracked and uncracked surfaces. This ensures precise data for training the model on specific crack patterns while minimizing the influence of environmental variables.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

TABLE V
DISTRIBUTION OF DOMAINS USED IN THE INVESTIGATION WITH ADDITIONAL STATISTICS

Domain Name	Crack	Uncrack	Train	Val	Test	Total	Avg. Crack Length (mm)	Image Original Resolution (px)
Bridge Deck	2025	11595	9806	2452	1362	13620	22.3	1024x768
Wall	3851	14287	13059	3265	1814	18138	25.1	1024x1024
Pavement	2608	21726	17520	4380	2434	24334	30.4	1280x720
SDNET2018	8484	47608	40385	10097	5610	56092	27.8	1024x1024
METU	20000	20000	28800	7200	4000	40000	28.5	1280x720
METU + SDNET2018	28484	67608	69185	17297	9610	96092	27.9	Mixed

3) METU+SDNET2018 Dataset: This dataset represents a domain we created by combining the METU and SDNET2018 datasets. The METU+SDNET2018 dataset comprises 96,092 images, including 28,484 cracked and 67,608 uncracked images. By combining the controlled, high-resolution data from METU with the real-world, diverse data from SDNET2018, this dataset provides a comprehensive resource for crack detection under varying structural and environmental conditions. The METU dataset contributes clear, consistent crack patterns, while SDNET2018 offers the variability needed to train the model for real-world applications.

Table V reports the six distinct domains redistributed from the abovementioned datasets. It details the distribution of samples across cracked, untracked training, validation, and testing subsets.

C. Evaluation Metrics

We performed experiments on each of the aforementioned domains. Thus, to assess the performance of our binary classification model in detecting concrete cracks, we utilize specific metrics obtained from the confusion matrix: true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN). These metrics quantitatively assess the model's performance and are computed using the following equations:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{7}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{8}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{9}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{10}$$

$$\text{AUC-ROC} = \int_0^1 \text{TPR}(t)\, d(\text{FPR}(t)) \tag{11}$$

Those metrics are essential for validating the model's utility in practical applications and ensuring its readiness for deployment in automated concrete inspection systems. In other words, High values in these metrics indicate strong model performance, showcasing its capacity to accurately and reliably distinguish between cracked and uncracked concrete images.

V. RESULTS AND DISCUSSION

This section details the results of the experiments evaluating the performance of binary models for concrete crack detection. Standard performance metrics, including Accuracy, Precision, Recall, F1-Score, and AUC-ROC, were used to assess the models. Additionally, we include a comparison of CrackVisionX with state-of-the-art approaches.

A. Performance Results Analysis

This subsection delves into our model's detailed performance analysis, examining its effectiveness in each of the six domains mentioned above through six different experiments. In other words, We evaluated ResNet50 (M1), MobileNet_v3_large (M2), DenseNet121 (M3) and EfficientNet-B0 (M4) on the Bridge Deck, Wall, Pavement, SDNET2018, METU, and METU + SDNET2018 domains, respectively, using different batch sizes (32, 64, and 128) and epoch settings (30 and 50).

1) Bridge Deck Domain: In this domain, EfficientNet-B0 consistently outperformed other models, as reported in Table VI. To be more specific, using a batch size of 32 and training for 50 epochs, EfficientNet-B0 achieved an accuracy, precision, recall, F1-score, and AUC-ROC of 99.71%, 99.74%, 99.74%, 99.74%, and 99.1%, respectively. Increasing the batch size to 128 with epochs of 50 resulted in an accuracy of 98.38%. These results indicate that EfficientNet-B0 slightly maintains high performance across all metrics.

2) Wall Domain: In this domain, EfficientNet-B0 and DenseNet121 showed robust performance, as shown in Table VII. In detail, EfficientNet-B0 achieved an accuracy, precision, recall, F1-score, and AUC-ROC of 99.78%, 99.72%, 100%, 99.86%, and 99.5%, respectively, with a batch size of 32 and trained for 50 epochs. On the other hand, DenseNet121 achieved notable results as well, achieving 99.56% accuracy, 99.72% precision, 99.72% recall, 99.72% F1-score, and 99.36% AUC-ROC, with the same batch size and number of epochs.

3) Pavement Domain: In this domain, EfficientNet-B0 showed outstanding performance across various configurations, as summarised in Table VIII. With a batch size of 32 and epochs of 50, the model achieved remarkable metrics: 99.55% accuracy, 99.77% precision, 99.68% recall, 99.72% F1-score, and an AUC-ROC of 98.87%. Moreover, when increasing the size of the batch to 64 with the same number of epochs and same model, EfficientNet-B0 achieved resulted in 99.22% accuracy, 99.36% precision, 99.77% recall, 99.56% F1-score, and an AUC-ROC of 97.18%. These metrics collectively demonstrate EfficientNet-B0's robustness and effectiveness in classifying pavement images.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
PERFORMANCE METRICS OF BRIDGE DECK CLASSIFICATION MODELS

Batch Size	Model	Epochs = 30					Epochs = 50				
		Accuracy	Precision	Recall	F1-score	AUC-ROC	Accuracy	Precision	Recall	F1-score	AUC-ROC
32	M1	98.31	99.23	98.8	99.01	97.09	98.9	99.57	99.14	99.36	98.29
	M2	98.6	98.48	99.91	99.19	95.34	99.12	98.98	100	99.49	96.92
	M3	98.97	99.32	99.49	99.4	97.69	99.49	99.83	99.57	99.7	99.27
	M4	99.56	99.66	100	99.83	98.97	99.71	99.74	99.74	99.74	99.1
64	M1	97.58	97.89	99.31	98.6	93.25	97.21	97.24	99.57	98.39	91.32
	M2	98.83	98.65	100	99.32	95.9	98.97	98.81	100	99.4	96.41
	M3	99.49	99.66	99.74	99.7	98.85	99.19	99.91	99.14	99.53	99.32
	M4	99.12	98.98	100	99.49	96.92	99.19	99.07	100	99.53	98.6
128	M1	96.7	97.41	98.81	98.1	90.98	98.38	98.17	100	99.07	94.02
	M2	98.38	98.33	99.83	99.07	94.48	98.31	98.17	99.92	99.03	93.98
	M3	97.8	97.57	99.91	98.73	92.52	98.9	98.73	100	99.36	96.15
	M4	99.27	99.16	100	99.58	97.28	98.38	98.25	99.92	99.07	94.25

^{M1} ResNet50, ^{M2} MobileNet_v3_large, ^{M3} DenseNet121, ^{M4} EfficientNetB0

TABLE VII
PERFORMANCE METRICS OF WALL CLASSIFICATION MODELS

Batch_Size	Model	Epochs=30					Epochs=50				
		Accuracy	Precision	Recall	F1-score	AUC-ROC	Accuracy	Precision	Recall	F1-score	AUC-ROC
32	M1	97.41	97.36	99.36	98.35	94.98	98.73	99.43	98.94	99.18	98.48
	M2	98.73	98.46	99.93	99.19	97.24	98.57	98.32	99.86	99.09	96.96
	M3	98.73	99.5	98.87	99.18	98.57	99.56	99.72	99.72	99.72	99.36
	M4	98.84	98.53	100	99.26	97.4	99.78	99.72	100	99.86	99.5
64	M1	95.98	95.2	99.86	97.47	91.14	96.03	95.88	99.15	97.49	92.15
	M2	98.35	97.92	100	98.95	96.29	98.57	98.19	100	99.09	96.78
	M3	99.06	99.64	99.15	99.4	98.96	98.07	98.31	99.22	98.76	96.64
	M4	98.9	98.6	100	99.3	97.52	97.74	97.3	99.86	98.56	95.1
128	M1	97.13	96.96	99.43	98.18	94.27	97.3	97.36	99.22	98.28	94.91
	M2	98.62	98.26	100	99.12	96.91	98.02	97.64	99.86	98.74	95.72
	M3	99.56	99.58	99.86	99.72	99.19	98.02	97.58	99.93	98.74	95.63
	M4	98.84	98.53	100	99.26	97.4	98.68	98.33	100	99.16	97.03

^{M1} ResNet50, ^{M2} MobileNet_v3_large, ^{M3} DenseNet121, ^{M4} EfficientNetB0

TABLE VIII
PERFORMANCE METRICS OF PAVEMENT CLASSIFICATION MODELS

Batch_Size	Model	Epochs=30					Epochs=50				
		Accuracy	Precision	Recall	F1-score	AUC-ROC	Accuracy	Precision	Recall	F1-score	AUC-ROC
32	M1	97.7	98.18	99.26	98.72	91.91	99.14	99.86	99.17	99.52	99.01
	M2	98.97	99.00	99.86	99.43	95.68	97.90	98.63	99.03	98.83	93.73
	M3	98.81	99.13	99.54	99.33	96.10	99.22	99.54	99.59	99.56	97.86
	M4	99.51	99.77	99.68	99.72	98.39	99.55	99.77	99.68	99.72	98.87
64	M1	98.03	98.32	99.49	98.90	92.60	97.37	98.17	98.90	98.53	91.73
	M2	99.06	99.09	99.86	99.47	96.07	98.93	98.86	99.95	99.41	95.15
	M3	98.23	98.54	99.49	99.02	93.57	98.03	98.54	99.26	98.90	93.45
	M4	98.85	98.95	99.77	99.36	95.44	99.22	99.36	99.77	99.56	97.18
128	M1	97.37	98.26	98.80	98.53	92.07	97.49	98.09	99.13	98.61	91.46
	M2	99.30	99.27	99.95	99.61	96.89	98.93	99.04	99.77	99.40	95.83
	M3	98.15	98.45	99.49	98.97	93.18	98.36	98.63	99.54	99.08	93.98
	M4	99.10	99.27	99.72	99.50	96.77	99.10	99.13	99.86	99.50	96.26

^{M1} ResNet50, ^{M2} MobileNet_v3_large, ^{M3} DenseNet121, ^{M4} EfficientNetB0

4) SDNET2018 Domain: In this domain, EfficientNet-B0 consistently exhibited superior performance across a range of configurations, as shown in Table IX. Specifically, with a batch size of 32 and epochs of 50, the model achieved an accuracy

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

TABLE IX
PERFORMANCE METRICS OF DOMAIN SDNET2018 CLASSIFICATION MODELS

Batch_Size	Model	Epochs=30					Epochs=50				
		Accuracy	Precision	Recall	F1-score	AUC-ROC	Accuracy	Precision	Recall	F1-score	AUC-ROC
32	M1	99.52	99.87	99.57	99.72	99.43	98.41	99.68	98.44	99.06	98.34
	M2	98.70	98.51	99.98	99.24	95.78	99.77	99.79	99.94	99.86	99.38
	M3	96.63	97.09	98.99	98.03	91.25	99.06	99.66	99.22	99.44	98.68
	M4	99.86	99.87	99.96	99.92	99.63	99.89	99.94	99.94	99.94	99.79
64	M1	98.27	98.20	99.79	98.99	94.81	97.99	97.74	99.94	98.82	93.54
	M2	99.14	99.00	100	99.50	97.19	98.97	98.79	100	99.39	96.61
	M3	99.77	99.75	99.98	99.86	99.29	99.41	99.43	99.87	99.65	98.36
	M4	99.14	99.00	100	99.50	97.19	98.88	98.71	99.98	99.34	96.36
128	M1	97.95	97.68	99.96	98.80	93.37	98.65	98.47	99.96	99.21	95.65
	M2	98.95	98.77	100	99.38	96.55	98.89	98.71	100	99.35	96.37
	M3	97.40	97.39	99.60	98.48	92.37	98.48	98.24	100	99.11	95.03
	M4	99.07	98.92	100	99.46	96.96	99.18	99.04	100	99.52	97.31

^{M1} ResNet50, ^{M2} MobileNet_v3_large, ^{M3} DenseNet121, ^{M4} EfficientNetB0

TABLE X
PERFORMANCE METRICS OF METU CLASSIFICATION MODELS

Batch_Size	Model	Epochs=30					Epochs=50				
		Accuracy	Precision	Recall	F1-score	AUC-ROC	Accuracy	Precision	Recall	F1-score	AUC-ROC
32	M1	99.88	99.90	99.85	99.87	99.87	99.90	99.90	99.90	99.90	99.90
	M2	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00	100.00
	M3	99.98	100.00	99.95	99.97	99.97	99.90	99.90	99.90	99.90	99.90
	M4	99.98	100.00	99.95	99.97	99.97	99.98	99.95	100.00	99.97	99.98
64	M1	99.90	99.95	99.98	99.90	99.90	99.75	99.65	99.85	99.75	99.75
	M2	100.00	100.00	100.00	100.00	100.00	99.98	99.95	100.00	99.97	99.98
	M3	99.88	99.90	99.85	99.87	99.87	99.92	99.95	99.90	99.92	99.92
	M4	99.92	99.85	100.00	99.92	99.93	99.92	99.95	99.90	99.92	99.92
128	M1	99.85	99.85	99.85	99.85	99.85	99.95	99.95	99.95	99.95	99.70
	M2	99.90	99.95	99.85	99.90	99.90	100.00	100.00	100.00	100.00	100.00
	M3	99.90	99.95	99.85	99.90	99.90	99.90	99.95	99.85	99.90	99.90
	M4	99.98	99.95	100.00	99.97	99.98	99.95	99.95	99.95	99.95	99.70

^{M1} ResNet50, ^{M2} MobileNet_v3_large, ^{M3} DenseNet121, ^{M4} EfficientNetB0

of 99.89%, a precision of 99.94%, a recall of 99.94%, an F1-score of 99.94%, and an AUC-ROC of 99.79%. From another perspective, when examining the model by using a batch size of 128 and 50 epochs, EfficientNet-B0 achieved an accuracy of 99.18%, a precision of 99.04%, a recall of 100%, an F1-score of 99.52%, and an AUC-ROC of 97.31%.

5) METU Domain: For the METU domain, MobileNetV3-Large and EfficientNet-B0 both demonstrated outstanding performance across various configurations, as shown in Table X. In detail, with a batch size of 32 and 50 epochs, MobileNetV3-Large and EfficientNet-B0 achieved exceptional results, including accuracies of 100% and 99.98%, precisions of 100% and 99.95%, recall of 100% and 100%, F1-scores of 100% and 99.97%, and AUC-ROCs of 100% and 99.98%, respectively. Additionally, with an increased batch size of 128, the models maintained high performance. MobileNetV3-Large and EfficientNet-B0 achieved accuracies of 100% and 99.95%, precisions of 100% and 99.95%, recall of 100% and 99.95%, F1-scores of 100% and 99.95%, and AUC-ROCs of 100% and 99.7%, respectively.

6) METU + SDNET2018 Domain: Here, The performance of EfficientNet-B0 also outperformed other models across different configurations, as reported in Table XI. More accurately, with a batch size of 32 and 50 epochs,EfficientNet-B0 achieved 99.92% accuracy, 99.94% precision, 99.94% recall, 99.94% F1-score, and a 99.9% AUC-ROC. Furthermore, with a batch size elevated to 128 and the same number of epochs, the model preserved high-performance levels. EfficientNet-B0 achieved 99.72% accuracy, 99.62% precision, 99.99% recall, 99.8% F1-score, and a 99.54% AUC-ROC.

Overall, EfficientNet-B0 showed exceptional performance across all domains and configurations, making it highly

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
PERFORMANCE METRICS OF DOMAIN METU AND SDNET2018 CLASSIFICATION MODELS

Batch_Size	Model	Epochs=30					Epochs=50				
		Accuracy	Precision	Recall	F1-score	AUC-ROC	Accuracy	Precision	Recall	F1-score	AUC-ROC
32	M1	99.32	99.88	99.16	99.52	99.44	99.06	99.75	98.92	99.33	99.16
	M2	99.84	99.87	99.91	99.89	99.80	99.66	99.60	99.91	99.76	99.48
	M3	99.43	99.84	99.35	99.59	99.48	99.41	99.79	99.36	99.58	99.44
	M4	99.88	99.96	99.87	99.91	99.88	99.92	99.94	99.94	99.94	99.90
64	M1	98.22	98.05	99.45	98.74	97.38	98.76	99.33	98.91	99.12	98.66
	M2	99.29	99.23	99.76	99.50	98.97	99.29	99.13	99.87	99.50	98.90
	M3	99.24	99.87	99.05	99.46	99.37	99.46	99.76	99.47	99.61	99.45
	M4	99.91	99.88	99.99	99.93	99.85	99.63	99.51	99.96	99.73	99.40
128	M1	98.16	97.93	99.48	98.70	97.25	98.48	98.36	99.50	98.93	97.78
	M2	99.32	99.13	99.91	99.52	98.92	99.17	98.87	99.96	99.41	98.63
	M3	98.69	98.61	99.54	99.07	98.10	98.45	98.36	99.45	98.90	97.76
	M4	99.84	99.81	99.97	99.89	99.76	99.72	99.62	99.99	99.80	99.54

^{M1} ResNet50, ^{M2} MobileNet_v3_large, ^{M3} DenseNet121, ^{M4} EfficientNetB0

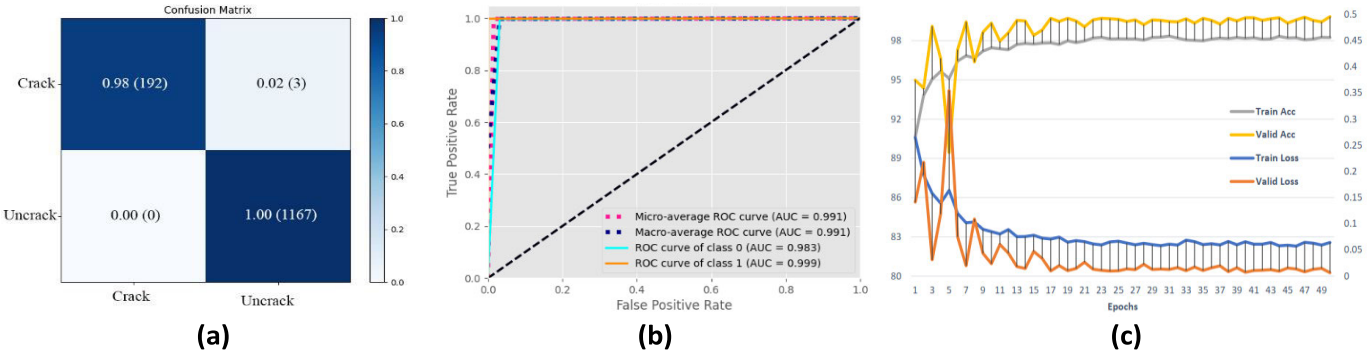


Fig. 5. Experiment 1: EfficientNet-B0 for bridge deck classification. Batch size: 32, Epochs: 50. Shown are (a) Normalized confusion matrix, (b) ROC curve, and (c) Training and validation accuracy.

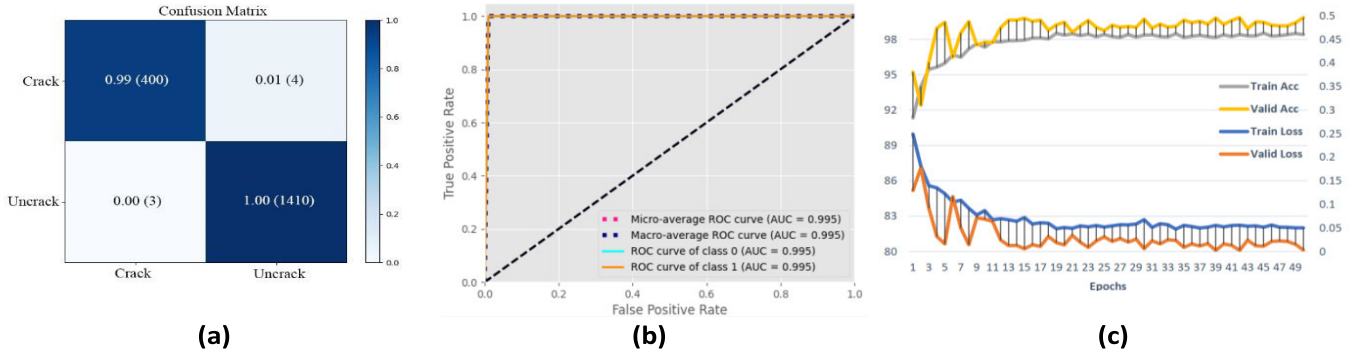


Fig. 6. Experiment 2: EfficientNet-B0 for wall classification. Batch size: 32, Epochs: 50. Shown are (a) Normalized confusion matrix, (b) ROC curve, and (c) Training and validation accuracy.

suitable for crack detection in various applications such as surveillance, construction, and maintenance. This consistent outperformance underscores EfficientNet-B0's robustness and efficacy in practical implementations for infrastructure monitoring. Consequently, we selected EfficientNet-B0 to provide more detailed performance metrics, including confusion matrices, ROC curves, and loss and accuracy of training and validation.

The performance of EfficientNet-B0 in experiments 1, 2, 3, 4, 5, and 6 for the Bridge Deck, Wall, Pavement, SDNET2018, METU, and METU + SDNET2018 domains, respectively, is illustrated in Figs. 5, 6, 7, 8, 9, and 10. More precisely, subfigures 5(a), 6(a), 7(a), 8(a), 9(a), and 10(a) display the confusion matrix, which reflects the confirmation of a high true positive rate for Bridge Deck, Wall, Pavement, SDNET2018, METU, and METU + SDNET2018 images, respectively.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

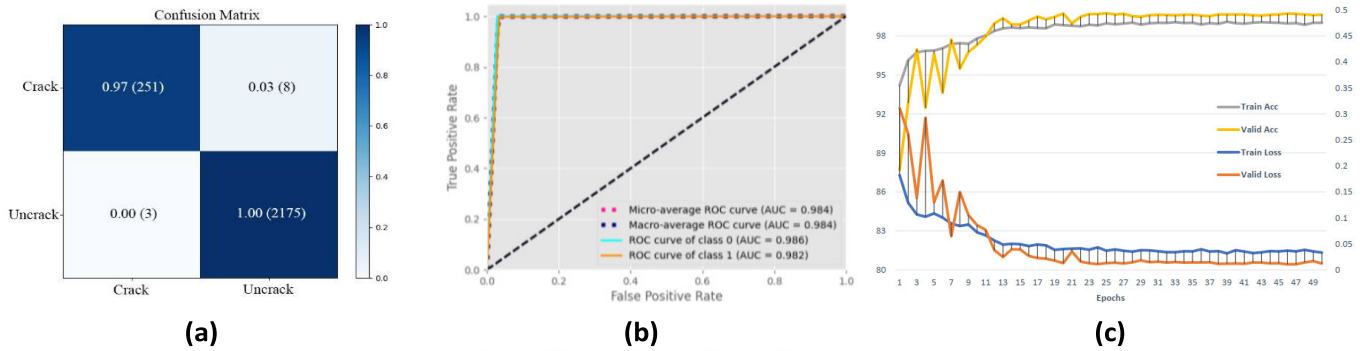


Fig. 7. Experiment 3: EfficientNet-B0 for pavement classification. Batch size: 32, Epochs: 50. Shown are (a) Normalized confusion matrix, (b) ROC curve, and (c) Training and validation accuracy.

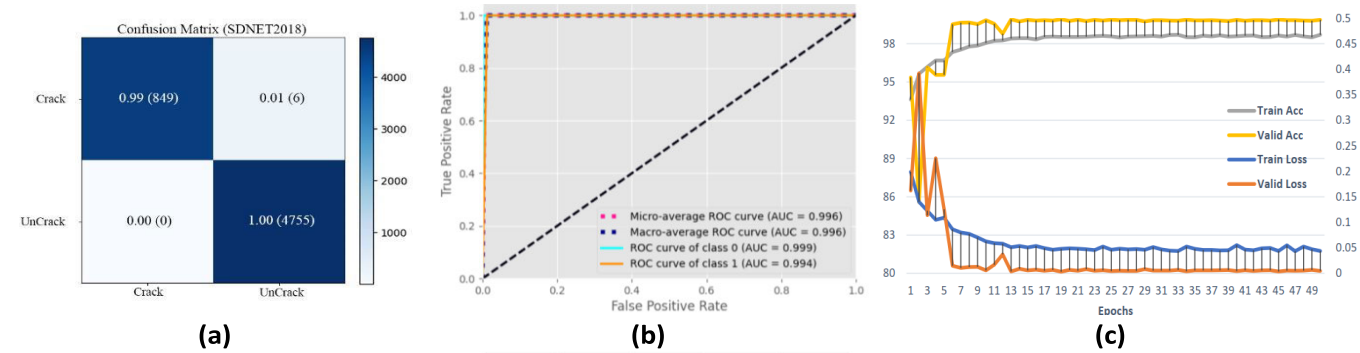


Fig. 8. Experiment 4: EfficientNet-B0 for SDNET2018 classification. Batch size: 32, Epochs: 50. Shown are (a) Normalized confusion matrix, (b) ROC curve, and (c) Training and validation accuracy.

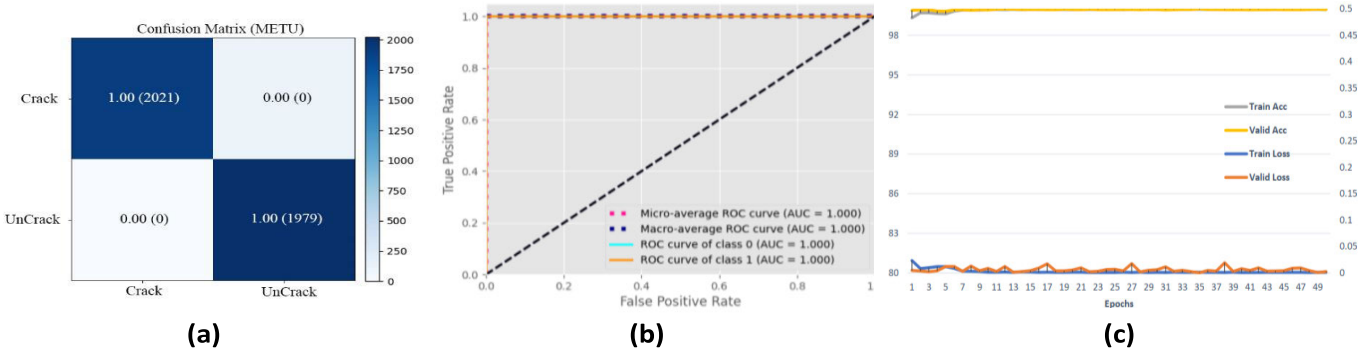


Fig. 9. Experiment 5: EfficientNet-B0 for METU classification. Batch size: 32, Epochs: 50. Shown are (a) Normalized confusion matrix, (b) ROC curve, and (c) Training and validation accuracy.

In addition, subfigures subfigures 5(b), 6(b), 7(b), 8(b), 9(b), and 10(b) present the ROC curves for the six aforementioned domains. These subfigures demonstrate an effective balance between precision and recall, with high areas under the curve indicating excellent predictive quality. Furthermore, subfigures 5(c), 6(c), 7(c), 8(c), 9(c), and 10(c) depict the training and validation performance for the six aforementioned domains. These subfigures illustrate remarkable improvements in accuracy and reductions in loss throughout the epochs, indicating robust learning and generalization.

B. Comparative Analysis of Crack Detection

In this section, we conducted a theoretically analyze and compared the CrackVisionX model with state-of-the-art models, including AlexNet [18], GoogleNet [18], ResNet18 [18], AlexNet [41], ResNet50 [41], VGG19 [42], CSDNet [43], Inception [44], ResNet [44], CNN [45], QWT, CNN_0FC [46], and Adaptive DBN [47]. This comparison was performed across six different domains:Bridge Deck, Wall, Pavement, SDNET2018, METU, and METU+SDNET2018, for each model. The comparison table in Table XII provides insights into the performance of those models.

To elaborate, Initially, deep learning models such as AlexNet [18], GoogleNet [18], and ResNet18 [18] demonstrate promising performance across multiple domains, such as Bridge Deck, Wall, and Pavement, achieving high accuracy and showcasing their effectiveness in image classification tasks. Then, the studies by Arbaoui et al. [41] highlight the performance of AlexNet and ResNet50 in the SDNET2018 domain, with ResNet50 outperforming AlexNet. After that,

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

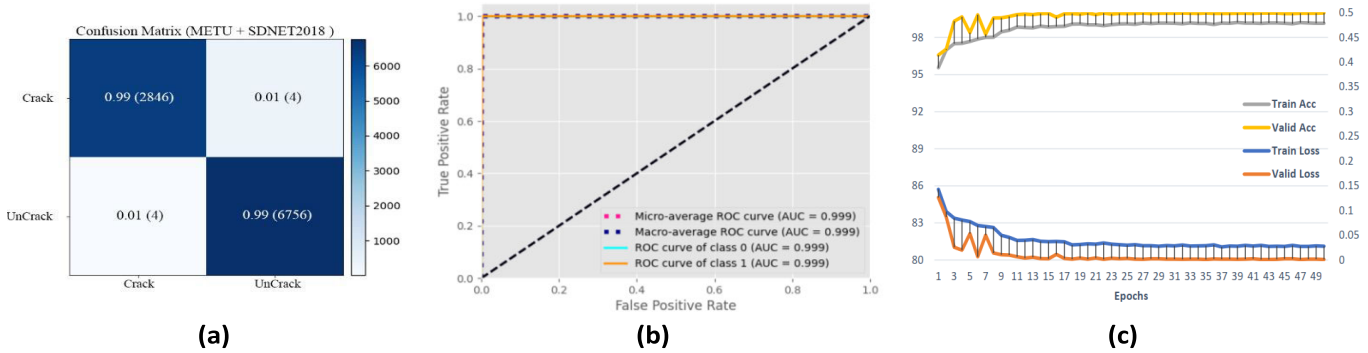


Fig. 10. Experiment 6: EfficientNet-B0 for METU + SDNET2018 classification. Batch size: 32, Epochs: 50. Shown are (a) Normalized confusion matrix, (b) ROC curve, and (c) Training and validation accuracy.

TABLE XII
COMPARISON OF THE CRACKVISIONX WITH STATE-OF-THE-ART APPROACHES

Domains	Epochs	Bridge Deck(%)	Wall(%)	Pavement(%)	SDNET2018 (%)	METU(%)	METU+SDNET2018(%)
Models (Accuracy)							
AlexNet [18]	D=10,W=9,P=10	91.92	89.31	95.52	-	-	-
GoogleLeNet [18]	D=10,W=9,P=10	92.95	91.95	95.56	-	-	-
Resnet18 [18]	D=10,W=9,P=10	92.19	91.9	95.56	-	-	-
AlexNet [41]	-	-	-	-	91.82	-	-
ResNet50 [41]	-	-	-	-	96.91	-	-
VGG19 [42]	100	-	-	-	91.76	-	-
CSDNet [43]	-	-	-	-	-	99.85	-
Inception [44]	-	-	-	-	-	99.8	-
ResNet [44]	-	-	-	-	-	97	-
CNN [45]	-	-	-	-	-	99.63	-
QWT [46]	50	98.33	98.33	98.67	98.44	99.8	-
CNN_0FC [46]	50	93.33	89.67	94	92.33	99.65	-
Adaptive DBN [47]	-	99.7	99.7	99.4	-	-	-
Ours Models (CrackVisionX)							
Resnet50	50	98.9	98.73	99.14	99.52	99.9	99.32
Mobilenet_v3_large	50	99.12	98.73	99.3	99.14	100	99.84
Densenet121	50	99.49	99.56	99.22	99.77	99.9	99.46
EfficientNetB0	50	99.71	99.78	99.55	99.89	99.98	99.92

Zeeshan et al. [42] present the performance of VGG19 in the METU domain, achieving competitive accuracy and indicating its effectiveness in classifying images in this specific domain. Subsequently, Bhattacharya et al. [43] showcase the high accuracy achieved by CSDNet in the METU domain, demonstrating the effectiveness of their proposed model for image classification. Following that, Kim et al. [44] present the performance of Inception and ResNet models in the METU domain, with Inception slightly outperforming ResNet. Later, Wijaya et al. [45] present the performance of a CNN model in the METU domain, achieving high accuracy and supporting the suitability of deep learning models for image classification in this domain. Thereafter, the QWT and CNN_0FC models proposed by R. Ahila Priyadharshini [46] achieve competitive accuracy in various domains, including Bridge Deck, Wall, and Pavement, METU, and SDNET2018 with Inception outperforming QWT. Finally, the Adaptive DBN model proposed by Kamada [47] achieves high accuracy in the Bridge Deck, Wall, and Pavement domains. In the end, our models (CrackVisionX) also achieve high accuracy across different domains. DenseNet121, ResNet50, MobileNet_v3_large, and EfficientNetB0 achieve competitive accuracy in the Bridge Deck, Wall, Pavement, SDNET2018, METU, and METU+SDNET2018 domains.

In addition, These results demonstrate the effectiveness of our models across several domains for diverse image classification tasks, as shown in Fig. 11. Overall, the comparison presented in Table XII and Fig. 11 provides valuable insights into the performance of different models in various domains, highlighting the accuracy of each model. This enables researchers and practitioners to select appropriate models for specific image classification tasks. Further research could focus on exploring the interpretability of these models, investigating their performance on larger datasets, and studying their generalizability to different domains. Additionally, model optimization techniques and ensemble methods could be explored to further improve the accuracy and robustness of the models in image classification tasks.

C. Mathematical and Efficiency Analysis

Table XIII demonstrates EfficientNet-B0s enhanced construction and recognition efficiency by comparing training time, inference speed, computational cost (FLOPs), and

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

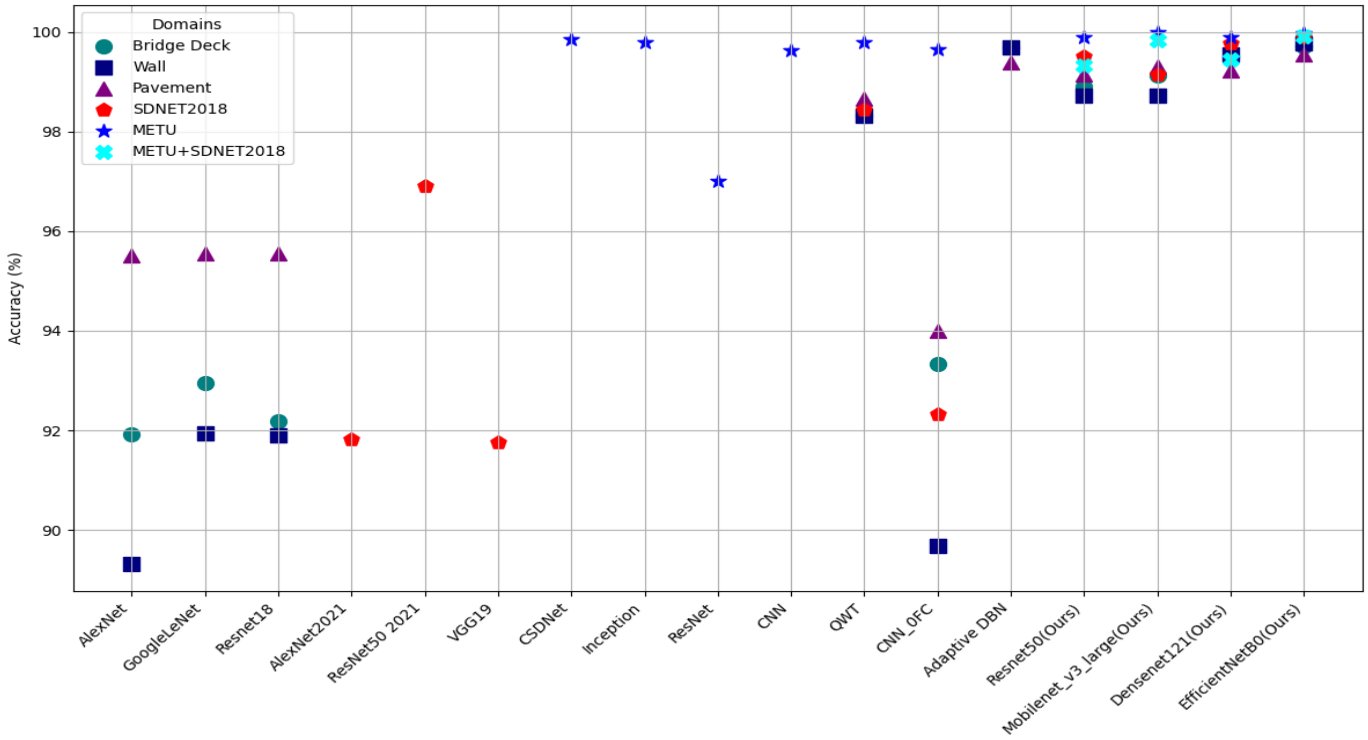


Fig. 11. Overall performance comparison of CrackVisionX with state-of-the-art approaches.

TABLE XIII
COMPARISON OF THE MODELS' EFFICIENCY FOR METU + SDNET2018

Model	FLOPs	Parameters (M)	Inference Time (ms)
ResNet50	4.1	27.7	26.94
MobileNetV3-Large	0.24	5.8	14.81
DenseNet121	2.9	8.0	49.89
EfficientNet-B0	0.39	5.6	29.19

memory usage, showcasing its suitability for deployment in resource-constrained environments.

1) Mathematical Superiority through Compound Scaling: EfficientNet-B0 achieves its performance by employing a compound scaling method that balances network depth (d), width (w), and resolution (r) as:

$$d = \alpha^{\phi}, \quad w = \beta^{\phi}, \quad r = \gamma^{\phi}$$

where α, β, γ are constants optimized through grid search, and ϕ represents the scaling factor. Unlike traditional architectures that scale only one dimension, EfficientNet-B0 optimizes resource allocation across all dimensions, achieving high accuracy with minimal computational cost.

2) Computational Cost (FLOPs): Floating Point Operations (FLOPs) measure the computational cost of a model. For a convolutional layer:

$$\text{FLOPs} = 2 \times H \times W \times C_{\text{in}} \times C_{\text{out}} \times K_h \times K_w$$

EfficientNet-B0 achieves significantly fewer FLOPs (0.39 billion) compared to ResNet50 (4.1 billion) and DenseNet121 (2.9 billion), as shown in Table XIII, reducing computational overhead.

3) Parameter Efficiency: EfficientNet-B0s compact architecture uses only 5.6 million parameters, compared to ResNet50 (27.7M) and DenseNet121 (8.0M). The parameter count is calculated as:

$$\text{Parameters} = (K_h \times K_w \times C_{\text{in}} \times C_{\text{out}}) + C_{\text{out}}$$

This parameter efficiency minimizes memory usage and makes EfficientNet-B0 suitable for resource-constrained devices.

4) Efficiency in Construction and Recognition: EfficientNet-B0 demonstrates exceptional efficiency in both construction and recognition tasks, balancing speed and precision for real-time applications.

a) Construction efficiency: EfficientNet-B0s 5.6 million parameters design minimizes memory usage and processing demands, enabling deployment on resource-limited platforms such as IoT devices and mobile systems.

b) Recognition efficiency: EfficientNet-B0 achieves an accuracy of 99.92% on the combined METU + SDNET2018 datasets, with an inference latency of 29.19 ms per image. These metrics demonstrate its suitability for real-time crack detection tasks.

c) Squeeze-and-Excitation (SE) blocks: EfficientNet-B0 uses SE blocks to prioritize critical features without additional computational costs as follows:

$$\text{SE}(X) = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot \text{GlobalAveragePool}(X)))$$

where W_1 and W_2 are learnable weights, and σ is the sigmoid activation.

EfficientNet-B0 combines computational efficiency, high accuracy, and fast inference speeds, making it ideal for IoT-based monitoring systems, mobile platforms, and

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

TABLE XIV
PERFORMANCE OF VARIOUS MODELS WITH AND WITHOUT DATA AUGMENTATION ON METU + SDNET2018 DOMAINS (BATCH SIZE: 32, EPOCHS: 50)

Model	Augmentation	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	AUC (%)
ResNet50	Without (✗)	95.63	96.23	97.60	96.91	94.30
	With (✓)	99.06	99.75	98.92	99.33	99.16
MobileNetV3-Large	Without (✗)	96.37	96.93	97.94	97.43	95.32
	With (✓)	99.66	99.60	99.91	99.76	99.48
DenseNet121	Without (✗)	96.19	96.69	97.92	97.30	95.02
	With (✓)	99.41	99.79	99.36	99.58	99.44
EfficientNet-B0	Without (✗)	96.62	96.78	98.46	97.61	95.37
	With (✓)	99.92	99.94	99.94	99.94	99.90

autonomous infrastructure inspections. Its innovative compound scaling and parameter efficiency enable superior recognition and construction performance, positioning it as a robust solution for real-world crack detection applications.

D. Impact of Data Augmentation Techniques

We performed a comparative analysis of models trained with and without data augmentation, considering both quantitative performance metrics and visual representations. Specifically, we evaluated the impact of data augmentation (including flips, rotations, and color adjustments) on model performance across two primary scenarios.

1) Evaluation of All Models with METU + SDNET2018 domain: In the first analysis, we examined the impact of data augmentation using four different models: EfficientNet-B0, ResNet50, DenseNet121, and MobileNetV3-Large. These models were trained on the combined METU + SDNET2018 domain under two configurations: (1) without augmentation (only resizing and normalization) and (2) with augmentation, which included transformations such as random flips, rotations, color jitter, and grayscale conversions. All models were trained with a batch size of 32 for 50 epochs.

Table XIV shows the performance metrics for models trained with and without augmentation. Data augmentation significantly enhanced performance across all metrics. For instance, EfficientNet-B0 achieved an accuracy of 96.62% without augmentation, which improved to 99.92% with augmentation. Similarly, DenseNet121 showed an F1-score increase from 97.30% to 99.58%.

Fig. 12 illustrates the ROC curves for models trained with and without augmentation. These results highlight that augmentation significantly improves AUC, enhancing model robustness and generalization to unseen data.

2) Evaluation of the EfficientNet-B0 Model Across All Domains: In the second scenario, we assessed the impact of data augmentation on individual domains (Bridge Deck, Wall, Pavement, SDNET2018, and METU) using the EfficientNet-B0 model. Each domain was trained for 50 epochs with a batch size of 32 under two configurations: (1) without augmentation (resizing and normalization only) and (2) with augmentation, including transformations such as random flips, rotations, color jitter, and grayscale conversions.

Table XV presents the results, showing significant performance improvements across all domains. For instance,

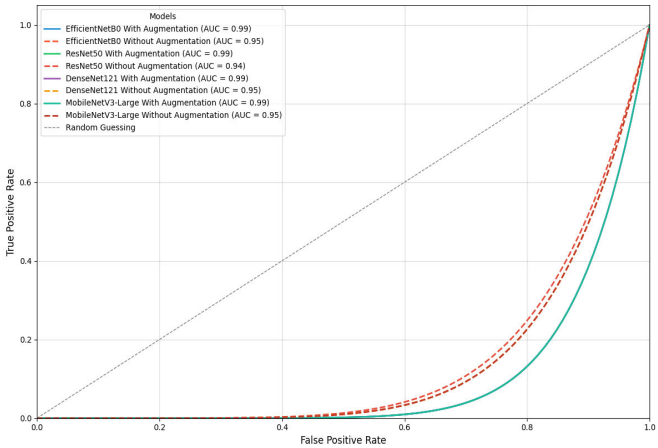


Fig. 12. ROC curves of models with and without data augmentation on METU + SDNET2018 domains (Batch Size: 32, Epochs: 50).

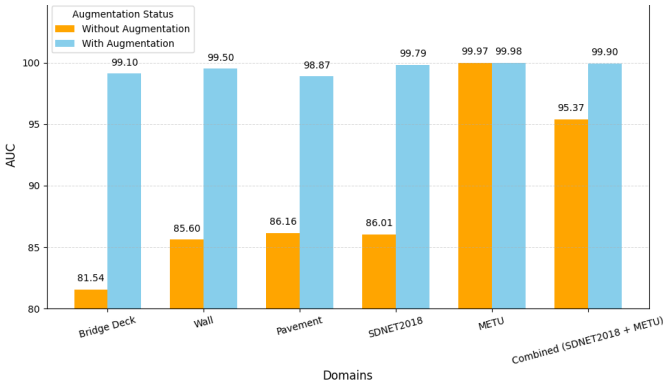


Fig. 13. AUC comparison of EfficientNet-B0 with and without augmentation across all domains (Batch size: 32, Epochs: 50).

accuracy on the Bridge Deck dataset increased from 92.95% to 99.71% after augmentation, with similar trends observed for other domains.

Fig. 13 shows the ROC curves for EfficientNet-B0 on specific domains with and without augmentation. Improved AUC scores confirm that data augmentation enhances generalization across all domain-specific datasets.

The overall findings from both evaluations demonstrate that data augmentation techniques significantly enhance model performance, particularly in handling unseen variations in test data. By mitigating overfitting and improving generalization,

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

TABLE XV
PERFORMANCE OF EFFICIENTNET-B0 WITH AND WITHOUT DATA AUGMENTATION ACROSS DIFFERENT DOMAINS (BATCH SIZE: 32, EPOCHS: 50)

Domain	Augmentation	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)	AUC (%)
Bridge Deck	Without (✗)	92.95	93.93	98.01	95.93	81.54
	With (✓)	99.71	99.74	99.74	99.74	99.10
Wall	Without (✗)	91.84	92.99	96.81	94.86	85.60
	With (✓)	99.78	99.72	100.00	99.86	99.50
Pavement	Without (✗)	96.09	96.65	99.03	97.82	86.16
	With (✓)	99.55	99.77	99.68	99.72	98.87
SDNET2018	Without (✗)	94.36	95.80	97.71	96.75	86.01
	With (✓)	99.89	99.94	99.94	99.94	99.79
METU	Without (✗)	99.98	100.00	99.95	99.97	99.97
	With (✓)	99.98	99.95	100.00	99.97	99.98
METU + SDNET2018	Without (✗)	96.62	96.78	98.46	97.61	95.37
	With (✓)	99.92	99.94	99.94	99.94	99.90

TABLE XVI
PERFORMANCE EVALUATION UNDER SP NOISE CONDITIONS (EFFICIENTNET-B0, METU + SDNET2018)

SP Noise Intensity	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	AUC-ROC
0.1	89.09	88.93	89.09	88.89	0.8515
0.2	87.21	87.59	87.21	86.44	0.8010

TABLE XVII
PERFORMANCE EVALUATION UNDER MB NOISE CONDITIONS (EFFICIENTNET-B0, METU + SDNET2018)

MB Noise (Length, θ)	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	AUC-ROC
(5, 10°)	99.97	99.97	99.97	99.97	0.9998
(10, 15°)	99.50	99.50	99.50	99.50	0.9935
(10, 20°)	99.60	99.60	99.60	99.60	0.9954
(20, 40°)	97.32	97.32	97.32	97.32	0.9684

these techniques make models more robust and better suited for deployment in diverse real-world scenarios.

E. Evaluation of the Proposed Method's Robustness Under Noise Effects

In practical scenarios, concrete surface images are often subjected to various types of noise during acquisition and transmission due to environmental conditions and sensor imperfections. Such disturbances can lead to information loss and distortions, posing challenges to the accuracy of crack detection methods. To ensure the robustness of the proposed method in real-world conditions, this study evaluates its performance under two prevalent noise types: Salt-and-Pepper (SP) Noise and Motion Blur (MB) Noise. SP noise simulates random pixel corruption caused by sensor imperfections, while MB noise replicates distortions due to relative motion during image capture.

The METU + SDNET2018 dataset was utilized for this evaluation, and EfficientNet-B0, known for its lightweight architecture and high classification efficiency, was selected as the best model for analysis. The following subsections comprehensively evaluate the model's performance under noisy conditions.

1) Effect of SP Noise on Model Performance: To evaluate the impact of SP noise, the model was tested on images contaminated with two levels of SP noise intensity: 0.1 and 0.2. Table XVI summarizes the results, presenting the model's accuracy, precision, recall, F1-score, and AUC-ROC under these conditions.

The results demonstrate that the model achieves robust performance even in the presence of noise. For a lower noise intensity (0.1), the model achieves an accuracy of 89.09% and an AUC-ROC of 0.8515. As the noise intensity increases to 0.2, a slight reduction in performance is observed, with the accuracy declining to 87.21% and the AUC-ROC to 0.8010. Despite this, all metrics remain above 80%, highlighting the model's resilience and adaptability to noisy environments. These findings are consistent with studies showing EfficientNet's robustness in various noisy conditions.

2) Effect of MB Noise on Model Performance: The motion blur noise is common in real-world applications, resulting from relative motion between the camera and focal position during image capture. To evaluate the robustness of the proposed method under such conditions, we tested the model on datasets contaminated with varying degrees of motion blur. Specifically, we considered motion length and motion angle of $(5, 10°)$, $(10, 15°)$, $(10, 20°)$, and $(20, 40°)$. Table XVII summarizes the evaluation metrics under these conditions, including accuracy, precision, recall, F1-score, and AUC-ROC.

The robustness analysis demonstrates the resilience of the proposed method against common noise types encountered in real-world applications. Under SP noise with an intensity of 0.2, the model achieved an accuracy of 87.21%, while under MB noise characterized by $(20, 40°)$, it achieved 97.32%. These results underscore the reliability of the proposed framework for detecting cracks in concrete structures, even under adverse conditions.

Fig. 14 presents the confusion matrices corresponding to the performance of the proposed method under different noise intensities. For Salt-and-Pepper (SP) noise, as depicted in subfigures (a) and (b), the model exhibits strong classification capabilities with minimal false positives and false negatives, even as the noise intensity increases from 0.1 to 0.2. On the other hand, Subfigures (c) through (f) illustrate the model's performance under various levels of Motion Blur (MB) noise, ranging from mild distortions (5, 10°) to more severe

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

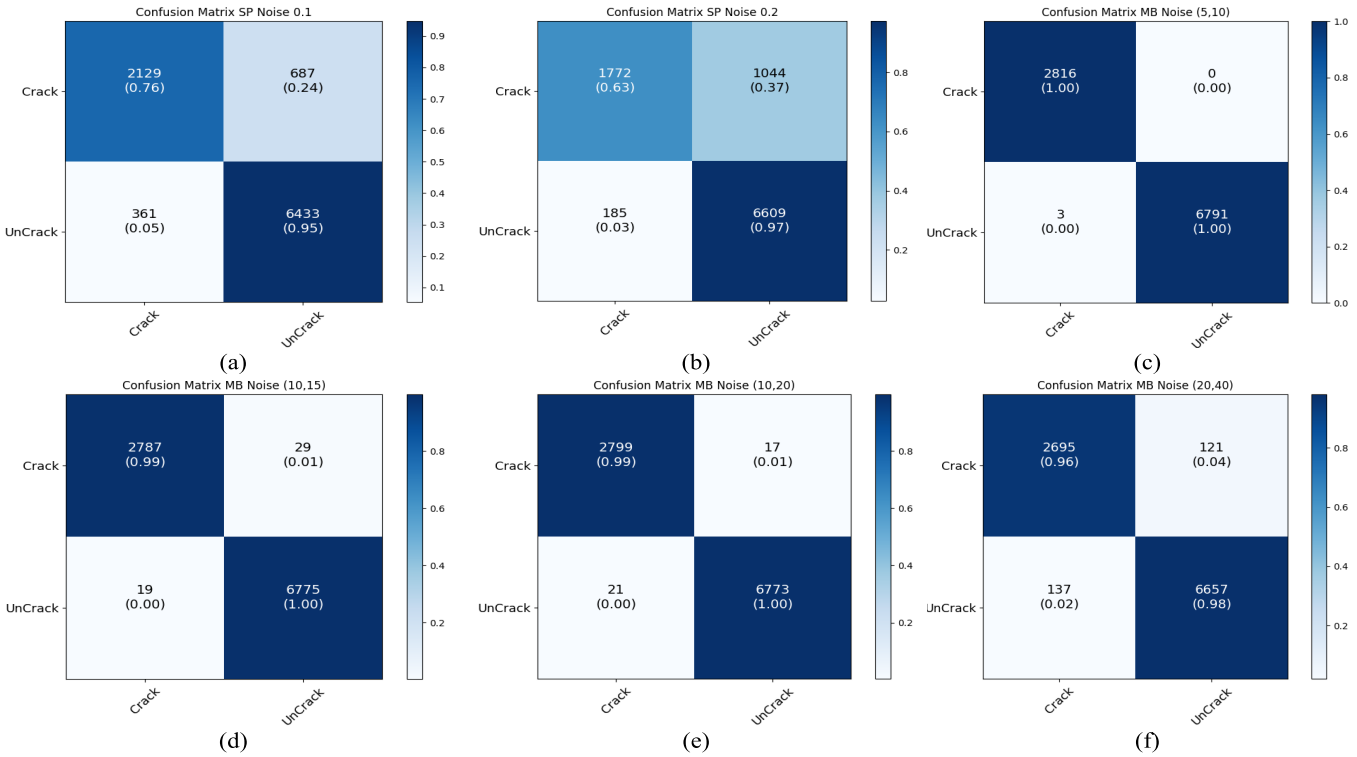


Fig. 14. Confusion matrices for model performance under different noise conditions: **(a)** SP noise with intensity 0.1, **(b)** SP noise with intensity 0.2, **(c)** MB noise with intensity (5, 10°), **(d)** MB noise with intensity (10, 15°), **(e)** MB noise with intensity (10, 20°), **(f)** MB noise with intensity (20, 40°). These matrices visually represent the model's classification performance under varying noise scenarios.

conditions (10, 20°) and (20, 40°). The results demonstrate that the model effectively manages light and moderate MB noise, with a noticeable but controlled increase in misclassifications under higher noise intensities.

VI. Conclusion

This paper introduces an advanced AI-based framework called CrackVisionX, designed to detect cracks in concrete structures. CrackVisionX employs CNNs augmented with fine-tuning techniques to accurately categorize input images as either cracked or non-cracked. By utilizing pre-trained CNN models and training them on new datasets of concrete cracks, CrackVisionX overcomes the limitations of manual and visual inspection methods, offering a more robust and efficient solution for concrete crack detection. Furthermore, the proposed model effectively addresses data imbalances and improves model robustness, achieving high accuracy rates in binary classification tasks by integrating data augmentation and crack extraction methods. Additionally, extensive preprocessing techniques were implemented on the METU and SDNET2018 datasets, creating six distinct domains: Bridge Deck, Wall, Pavement, SDNET2018, METU, and a combined METU + SDNET2018. We carried out a theoretical analysis and compared the CrackVisionX model with state-of-the-art models across six distinct domains. The results demonstrate the effectiveness of CrackVisionX for diverse image classification tasks and its superior accuracy compared to the other models. In the end, CrackVisionX not only advances the precision of crack detection but also sets a new standard for the application of AI in infrastructure safety and maintenance.

In future research, we aim to expand CrackVisionX beyond binary classification to include multi-class and multi-damage detection, enabling the identification and localization of various structural defects, such as spalling and corrosion, for more comprehensive structural health assessments. Additionally, we plan to adapt the framework to detect defects in diverse materials and infrastructure types, including asphalt pavements. Enhancing the model's ability to detect fine cracks in environments with high noise or low contrast will also be prioritized to improve sensitivity and accuracy in early-stage structural monitoring. Furthermore, we intend to deploy CrackVisionX in practical settings, such as real-time monitoring of concrete structures in bridges and pavements. These deployments will evaluate the model's robustness against variations in environmental conditions, lighting, and surface textures, thereby bridging the gap between experimental validation and real-world applicability. These advancements will address limitations and broaden CrackVisionX's applicability in infrastructure safety and maintenance.

References

[1] R. Ali, J. H. Chuah, M. S. A. Talip, N. Mokhtar, and M. A. Shoaib, "Structural crack detection using deep convolutional neural networks," *Autom. Construct.*, vol. 133, Jan. 2022, Art. no. 103989. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0926580521004404

[2] K. Li et al., "Rethinking lightweight convolutional neural networks for efficient and high-quality pavement crack detection," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 1, pp. 237–250, Jan. 2024.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

[3] Y. Hou, H. Shi, N. Chen, Z. Liu, H. Wei, and Q. Han, "Vision image monitoring on transportation infrastructures: A lightweight transfer learning approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 11, pp. 12888–12899, Nov. 2023.

[4] G. Li, Q. Liu, W. Ren, W. Qiao, B. Ma, and J. Wan, "Automatic recognition and analysis system of asphalt pavement cracks using interleaved low-rank group convolution hybrid deep network and SegNet fusing dense condition random field," *Measurement*, vol. 170, Jan. 2021, Art. no. 108693. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S026322412031201X

[5] J. Fang, B. Qu, and Y. Yuan, "Distribution equalization learning mechanism for road crack detection," *Neurocomputing*, vol. 424, pp. 193–204, Feb. 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID

[6] Z. Zheng, H. Qi, L. Zhuang, and Z. Zhang, "Automated rail surface crack analytics using deep data-driven models and transfer learning," *Sustain. Cities Soc.*, vol. 70, Jul. 2021, Art. no. 102898. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2210670721001864

[7] T. H. Dinh et al., "Toward vision-based concrete crack detection: Automatic simulation of real-world cracks," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–15, 2023.

[8] D. M. Frangopol and M. Liu, "Maintenance and management of civil infrastructure based on condition, safety, optimization, and life-cycle cost," *Struct. Infrastruct. Eng.*, vol. 3, no. 1, pp. 29–41, Mar. 2006.

[9] M. F. S. Rodrigues, J. M. C. Teixeira, and J. C. P. Cardoso, "Buildings envelope anomalies: A visual survey methodology," *Construct. Building Mater.*, vol. 25, no. 5, pp. 2741–2750, May 2011. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0950061810007075

[10] L. Zhang, Z. Wang, L. Wang, Z. Zhang, X. Chen, and L. Meng, "Machine learning-based real-time visible fatigue crack growth detection," *Digit. Commun. Netw.*, vol. 7, no. 4, pp. 551–558, Nov. 2021. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2352864821000146

[11] L. Guo-qing, "Automatic detection algorithm for expressway subgrade diseases based on SVM," *Zhongguo Gonglu Xuebao/China J. Highway Transp.*, vol. 26, pp. 42–47, Mar. 2013.

[12] Y. Hou et al., "The state-of-the-art review on applications of intrusive sensing, image processing techniques, and machine learning methods in pavement monitoring and analysis," *Engineering*, vol. 7, no. 6, pp. 845–856, Jun. 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID

[13] A. Al Smadi, S. Yang, Z. Kai, A. Mehmood, M. Wang, and A. Alsanabani, "Pansharpening based on convolutional autoencoder and multi-scale guided filter," *EURASIP J. Image Video Process.*, vol. 2021, no. 1, pp. 1–20, Dec. 2021.

[14] A. A. Smadi, S. Yang, A. Abugabah, A. A. Alzubi, and L. Sanzogni, "A pansharpening based on the non-subsampled contourlet transform and convolutional autoencoder: Application to QuickBird imagery," *IEEE Access*, vol. 10, pp. 44778–44788, 2022.

[15] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *Commun. ACM*, vol. 60, no. 6, pp. 84–90, May 2017.

[16] A. Al-Hinawi and R. Alelaimat, "Using deep learning for profitable concrete forecasting methods," *Int. Arab J. Inf. Technol.*, vol. 21, no. 5, pp. 832–843, 2024.

[17] A. A. Alkannad et al., "CrackVision: Effective concrete crack detection with deep learning and transfer learning," *IEEE Access*, vol. 13, pp. 29554–29576, 2025, doi: 10.1109/ACCESS.2025.3540841.

[18] N. Sharma, R. Dhir, and R. Rani, "Crack detection in concrete using transfer learning," *Adv. Math., Sci. J.*, vol. 9, no. 6, pp. 3895–3906, Jul. 2020.

[19] V. P. Golding, Z. Gharineiat, H. S. Munawar, and F. Ullah, "Crack detection in concrete structures using deep learning," *Sustainability*, vol. 14, no. 13, p. 8117, Jul. 2022. [Online]. Available: https://www.mdpi.com/2071-1050/14/13/8117

[20] H. Zhang, C. Liu, J. Ho, and Z. Zhang, "Crack detection based on convnext and normalization," *J. Phys., Conf.*, vol. 2289, no. 1, Jun. 2022, Art. no. 012022.

[21] T. Jiang et al., "Lightweight object detection network for multi-damage recognition of concrete bridges in complex environments," *Comput.-Aided Civil Infrastruct. Eng.*, vol. 39, no. 23, pp. 3646–3665, Apr. 2024.

[22] B. Guan and J. Li, "Lightweight detection network for bridge defects based on model pruning and knowledge distillation," *Structures*, vol. 62, Apr. 2024, Art. no. 106276. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2352012424004284

[23] Y. Yu, M. Rashidi, B. Samali, M. Mohammadi, T. N. Nguyen, and X. Zhou, "Crack detection of concrete structures using deep convolutional neural networks optimized by enhanced chicken swarm algorithm," *Struct. Health Monitor.*, vol. 21, no. 5, pp. 2244–2263, Sep. 2022. [Online]. Available: https://api.semanticscholar.org/CorpusID

[24] Y. Wan, Z. Liu, J. Wang, X. Gao, R. Xia, and H. Lyu, "Improved particle swarm optimization crack identification for building wall," in *Proc. China Autom. Congr. (CAC)*, 2022, pp. 660–665.

[25] D. Amirkhani, M. S. Allili, L. Hebbache, N. Hammouche, and J.-F. Lapointe, "Visual concrete bridge defect classification and detection using deep learning: A systematic review," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 9, pp. 10483–10505, Sep. 2024.

[26] F. Şermet and I. Pacal, "Deep learning approaches for autonomous crack detection in concrete wall, brick deck and pavement," *Dicle Niversitesi Mhendislik Fakltesi Mhendislik Dergisi*, vol. 15, no. 2, pp. 503–513, Apr. 2024. [Online]. Available: https://api.semanticscholar.org/CorpusID

[27] Z. A. Bukhsh, N. Jansen, and A. Saeed, "Damage detection using in-domain and cross-domain transfer learning," *Neural Comput. Appl.*, vol. 33, no. 24, pp. 16921–16936, Dec. 2021.

[28] D. Dais, İ. E. Bal, E. Smyrou, and V. Sarhosis, "Automatic crack classification and segmentation on masonry surfaces using convolutional neural networks and transfer learning," *Autom. Construction*, vol. 125, May 2021, Art. no. 103606. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0926580521000571

[29] Y. Jiang, D. Pang, and C. Li, "A deep learning approach for fast detection and classification of concrete damage," *Autom. Construct.*, vol. 128, Aug. 2021, Art. no. 103785.

[30] M. Żarski, B. Wójcik, K. Ksiąüek, and J. A. Miszczak, "Finicky transfer learning—A method of pruning convolutional neural networks for cracks classification on edge devices," *Comput.-Aided Civil Infrastruct. Eng.*, vol. 37, no. 4, pp. 500–515, Mar. 2022.

[31] S. Dorafshan, R. J. Thomas, and M. Maguire, "SDNET2018: An annotated image dataset for non-contact concrete crack detection using deep convolutional neural networks," *Data Brief*, vol. 21, pp. 1664–1668, Dec. 2018.

[32] K. Gopalakrishnan, S. K. Khaitan, A. Choudhary, and A. Agrawal, "Deep convolutional neural networks with transfer learning for computer vision-based data-driven pavement distress detection," *Construct. Building Mater.*, vol. 157, pp. 322–330, Dec. 2017.

[33] R. E. Philip, A. D. Andrushia, A. Nammalvar, B. G. A. Gurupatham, and K. Roy, "A comparative study on crack detection in concrete walls using transfer learning techniques," *J. Compos. Sci.*, vol. 7, no. 4, p. 169, Apr. 2023.

[34] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, vol. 16, Jun. 2016, pp. 770–778.

[35] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jul. 2017, pp. 2261–2269.

[36] A. Howard et al., "Searching for MobileNetV3," 2019, *arXiv:1905.02244*.

[37] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," 2019, *arXiv:1905.11946*.

[38] X. Glorot, A. Bordes, and Y. Bengio, "Deep sparse rectifier neural networks," in *Proc. 14th Int. Conf. Artif. Intell. Statist.*, 2011, pp. 315–323. [Online]. Available: https://proceedings.mlr.press/v15/glorot11a.html

[39] Ç. F. Özgenel, "Concrete crack images for classification," Mendeley Data, Middle East Tech. Univ. (METU), Ankara, Turkey, Tech. Rep. V2, 2019, doi: 10.17632/5y9wdsg2zt.2.

[40] Ç. F. Özgenel and A. G. Sorguç, "Performance comparison of pretrained convolutional neural networks on crack detection in buildings," in *Proc. 35th Int. Symp. Autom. Robot. Construction (ISARC)*, J. Teizer, Ed., Jul. 2018, pp. 693–700.

[41] A. Arbaoui, A. Ouahabi, S. Jacques, and M. Hamiane, "Concrete cracks detection and monitoring using deep learning-based multiresolution analysis," *Electronics*, vol. 10, no. 15, p. 1772, Jul. 2021. [Online]. Available: https://www.mdpi.com/2079-9292/10/15/1772

[42] M. Zeeshan, S. M. Adnan, W. Ahmad, and F. Z. Khan, "Structural crack detection and classification using deep convolutional neural network," *Pakistan J. Eng. Technol.*, vol. 4, no. 4, pp. 50–56, Dec. 2021.

[43] G. Bhattacharya, N. B. Puhan, and B. Mandal, "Kernelized dynamic convolution routing in spatial and channel interaction for attentive concrete defect recognition," *Signal Process., Image Commun.*, vol. 108, Oct. 2022, Art. no. 116818.

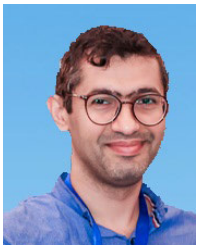
Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.

[44] B. Kim, N. Yuvaraj, K. R. Sri Preethaa, and R. A. Pandian, "Surface crack detection using deep learning with shallow CNN architecture for enhanced computation," *Neural Comput. Appl.*, vol. 33, no. 15, pp. 9289–9305, Aug. 2021. [Online]. Available: https://api.semanticscholar.org/CorpusID

[45] I. G. P. S. Wijaya, A. P. J. Dwitama, I. B. K. Widiartha, and S. A. Putra, "Classification of building cracks image using the convolutional neural network method," in *Proc. Int. Conf. Advancement Data Sci., E-Learning Inf. Syst. (ICADEIS)*, Oct. 2020, pp. 1–6.

[46] R. Ahila Priyadharshini, S. Arivazhagan, and M. Arun, "Crack recognition on concrete structures based on machine crafted and hand crafted features," *Exp. Syst. Appl.*, vol. 228, Oct. 2023, Art. no. 120447. [Online]. Available: https://api.semanticscholar.org/CorpusID

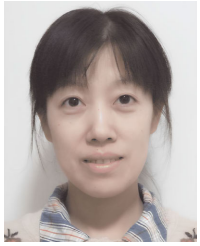
[47] S. Kamada, T. Ichimura, and T. Iwasaki, "An adaptive structural learning of deep belief network for image-based crack detection in concrete structures using SDNET2018," in *Proc. Int. Conf. Image Process. Robot. (ICIP)*, Mar. 2020, pp. 1–6.



Abdulrahman A. ALKannad received the B.S. degree in computer science from Al-Hodeidah University, Yemen, in 2010, and the M.S. degree in computer science and technology from Xidian University, China, in 2020, where he is currently pursuing the Ph.D. degree in computer science and technology. He was a Lecturer with National University, Yemen. His research interests include artificial intelligence, computer vision, machine learning, deep learning, and information technology.

Ahmad AL Smadi received the Ph.D. degree in computer science and technology from the School of Artificial Intelligence, Xidian University, Xi'an, China. He was a Research Assistant with the College of Technological Innovation, Zayed University, United Arab Emirates. He is currently an Assistant Professor with the Department of Data Science and Artificial Intelligence, Zarqa University. His research interests include information systems, computer vision, machine learning, and deep learning.

Moeen AL-Makhlafi received the B.S. degree in mathematical and computer science from Ibb University, Ibb, Yemen, in 2010, and the M.S. and Ph.D. degrees in computer science and technology from Xidian University, Xi'an, China, in 2019 and 2024, respectively. He is currently a Lecturer with Wuhan College, Wuhan, China. His main research interests include optical interconnect networks, data center networks, high-performance computing networks, and machine learning.



Shuyuan Yang (Senior Member, IEEE) received the B.S. degree in electrical engineering and the M.S. and Ph.D. degrees in circuit and systems from Xidian University, Xi'an, China, in 2000, 2003, and 2005, respectively. She is currently a Professor with the Key Laboratory of Intelligent Perception and Image Understanding of Ministry of Education, Xidian University. Her research interests include intelligent signal processing, machine learning, and image processing.

Zhixi Feng (Member, IEEE) received the B.S. degree in automation from Lanzhou University of Technology, Lanzhou, China, in 2012, and the Ph.D. degree in intelligent information processing from Xidian University, Xi'an, China, in 2018. He is currently an Associate Professor with the School of Artificial Intelligence, Xidian University. His research interests include intelligence information processing and machine learning.

Authorized licensed use limited to: Civil Aviation University of China. Downloaded on February 07,2026 at 12:21:24 UTC from IEEE Xplore. Restrictions apply.